The Concise TypeScript Book

Simone Poggiali

The Concise TypeScript Book

The Concise TypeScript Book memberikan gambaran menyeluruh dan ringkas tentang kemampuan TypeScript. Buku ini menyajikan penjelasan yang jelas dan mencakup semua aspek versi terbaru bahasa tersebut, mulai dari sistem tipenya yang andal hingga fitur-fitur tingkat lanjut.

Baik Anda seorang pemula maupun pengembang berpengalaman, buku ini merupakan sumber daya yang sangat berharga untuk meningkatkan pemahaman dan kemahiran Anda dalam TypeScript.

Buku ini sepenuhnya gratis dan bersifat sumber terbuka.

Saya percaya bahwa pendidikan teknis berkualitas tinggi harus dapat diakses oleh semua orang. Karena alasan ini, saya tetap menyediakan buku ini secara gratis dan memperbaruinya secara berkala dengan berbagai penyempurnaan dan contoh baru.

Temukan **The Concise TypeScript Book Plus Edition**.



Bagi pembaca yang ingin melangkah lebih jauh dari edisi sumber terbuka, **The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7** menyertakan konten tambahan dan eksklusif yang berfokus pada penerapan praktis.

Plus Edition mencakup:

- **Diperbarui untuk TypeScript 7** — pembahasan tentang fitur-fitur terbaru TypeScript 7 dan penyempurnaan bahasanya.
- **TypeScript dengan React** — panduan praktis untuk memberikan tipe pada komponen, props, hooks, event,

children, refs, dan pola-pola umum React.
- **Resep TypeScript untuk Proyek Dunia Nyata** — contoh-contoh terarah yang membahas masalah praktis yang dihadapi pengembang saat membangun dan memelihara aplikasi TypeScript.

Dengan membeli Plus Edition, Anda juga secara langsung mendukung pengembangan dan pemeliharaan berkelanjutan buku gratis yang bersifat sumber terbuka ini.

Plus Edition tersedia dalam bahasa Inggris dan Italia di Amazon di seluruh dunia. Jelajahi Plus Edition dan beli di Amazon.

Dukung Proyek Ini

Jika buku gratis ini membantu Anda memperbaiki bug, memahami konsep yang sulit, atau mengembangkan karier, pertimbangkan untuk mendukung karya saya dengan membayar sesuai keinginan Anda—kontribusi yang disarankan sebesar **$5**—atau mentraktir saya secangkir kopi.

Dukungan Anda membantu saya menjaga konten tetap mutakhir dan memperluasnya dengan contoh-contoh baru, penjelasan yang lebih jelas, serta panduan praktis tambahan.





Terjemahan

Buku ini telah diterjemahkan ke dalam beberapa bahasa, termasuk:

Bahasa Inggris

Bahasa Bulgaria

Bahasa Jerman

Bahasa Prancis

Bahasa Indonesia

Bahasa Italia

Bahasa Jepang

Bahasa Korea

Bahasa Polandia

Bahasa Portugis (Brasil)

Bahasa Swedia

Bahasa Turki

Vietnamese

Bahasa Tionghoa

Bahasa Spanyol

Bahasa Thai

Bahasa Rusia

Bahasa Arab Bahasa Ceko

Unduhan dan situs web

Anda juga dapat mengunduh versi EPUB:

https://github.com/gibbok/typescript-book/tree/main/downloads

Versi daring tersedia di:

https://gibbok.github.io/typescript-book

Daftar Isi

- The Concise TypeScript Book
  - Dukung Proyek Ini
  - Terjemahan
  - Unduhan dan situs web
  - Daftar Isi
  - Pendahuluan
  - Tentang Penulis
  - Pengenalan TypeScript
    - Apa itu TypeScript?
    - Mengapa TypeScript?
    - TypeScript dan JavaScript
    - Pembuatan Kode TypeScript
    - JavaScript Modern Saat Ini (Downleveling)
  - Memulai dengan TypeScript
    - Instalasi
    - Konfigurasi
    - Berkas Konfigurasi TypeScript
      - target
      - lib
      - strict

        - module
        - moduleResolution
        - esModuleInterop
        - jsx
        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - Saran Migrasi ke TypeScript
- Menjelajahi Sistem Tipe
    - Layanan Bahasa TypeScript
    - Pengetikan Struktural
    - Aturan Dasar Perbandingan TypeScript
    - Tipe sebagai Himpunan
    - Menetapkan tipe: Deklarasi Tipe dan Asersi Tipe
        - Deklarasi Tipe
        - Asersi Tipe
        - Deklarasi Ambient
    - Pemeriksaan Properti dan Pemeriksaan Properti Berlebih
    - Tipe Lemah
    - Pemeriksaan Ketat Object Literal (Freshness)
    - Inferensi Tipe
    - Inferensi Lebih Lanjut
    - Pelebaran Tipe
    - Const
        - Modifier Const pada Parameter Tipe
        - Asersi Const
    - Anotasi Tipe Eksplisit
    - Penyempitan Tipe
        - Kondisi

        - Melempar atau Mengembalikan
        - Union Terdiskriminasi
        - Type Guard Buatan Pengguna
        - Penyempitan switch-true
    - Tipe Primitif
        - string
        - boolean
        - number
        - bigint
        - Symbol
        - null dan undefined
        - Array
        - any
    - Anotasi Tipe
    - Properti Opsional
    - Properti Readonly
    - Index Signature
    - Memperluas Tipe
    - Tipe Literal
    - Inferensi Literal
    - strictNullChecks
    - Enum
        - Enum numerik
        - Enum string
        - Enum konstan
        - Pemetaan balik
        - Enum ambient
        - Anggota terkomputasi dan konstan
    - Penyempitan
        - Type guard typeof
        - Penyempitan berdasarkan truthiness
        - Penyempitan berdasarkan kesetaraan

        - Penyempitan dengan Operator In
        - Penyempitan instanceof
    - Penetapan
    - Analisis Alur Kontrol
    - Predikat Tipe
    - Union Terdiskriminasi
    - Tipe never
    - Pemeriksaan Kelengkapan
    - Tipe Objek
    - Tipe Tuple (Anonim)
    - Tipe Tuple Bernama (Berlabel)
    - Tuple dengan Panjang Tetap
    - Tipe Union
    - Tipe Intersection
    - Pengindeksan Tipe
    - Tipe dari Nilai
    - Tipe dari Nilai Kembalian Fungsi
    - Tipe dari Modul
    - Mapped Type
    - Modifier Mapped Type
    - Tipe Kondisional
    - Tipe Kondisional Distributif
    - Inferensi Tipe infer dalam Tipe Kondisional
    - Tipe Kondisional yang Telah Didefinisikan
    - Tipe Union Template
    - Tipe Any
    - Tipe Unknown
    - Tipe Void
    - Tipe Never
    - Interface dan Type
        - Sintaks Umum
        - Tipe Dasar

        - Objek dan Interface
        - Tipe Union dan Intersection
    - Tipe Primitif Bawaan
    - Objek JS Bawaan yang Umum
    - Overload
    - Penggabungan dan Ekstensi
    - Perbedaan antara Type dan Interface
    - Class
        - Sintaks Umum Class
        - Constructor
        - Constructor Private dan Protected
        - Modifier Akses
        - Get dan Set
        - Auto-Accessor dalam Class
        - this
        - Properti Parameter
        - Class Abstrak
        - Dengan Generic
        - Decorator
            - Decorator Class
            - Decorator Properti
            - Decorator Metode
            - Decorator Getter dan Setter
            - Metadata Decorator
        - Pewarisan
        - Anggota Statis
        - Inisialisasi properti
        - Overload metode
    - Generic
        - Tipe Generic
        - Class Generic
        - Batasan Generic

        - Penyempitan kontekstual generic
    - Tipe Struktural yang Dihapus
    - Namespace
    - Symbol
    - Direktif Triple-Slash
    - Manipulasi Tipe
        - Membuat Tipe dari Tipe
        - Tipe Akses Terindeks
        - Tipe Utilitas
            - Awaited<T>
            - Partial<T>
            - Required<T>
            - Readonly<T>
            - Record<K, T>
            - Pick<T, K>
            - Omit<T, K>
            - Exclude<T, U>
            - Extract<T, U>
            - NonNullable<T>
            - Parameters<T>
            - ConstructorParameters<T>
            - ReturnType<T>
            - InstanceType<T>
            - ThisParameterType<T>
            - OmitThisParameter<T>
            - ThisType<T>
            - Uppercase<T>
            - Lowercase<T>
            - Capitalize<T>
            - Uncapitalize<T>
            - NoInfer<T>
    - Lain-lain

- Error dan Penanganan Exception
- Class Mixin
- Fitur Bahasa Asinkron
- Iterator dan Generator
- Referensi TsDocs JSDoc
- @types
- JSX
- Modul ES6
- Operator Eksponensiasi ES7
- Pernyataan for-await-of
- Meta-properti new target
- Ekspresi Import Dinamis
- “tsc –watch”
- Operator Asersi Non-null
- Deklarasi dengan nilai default
- Optional Chaining
- Operator Nullish Coalescing
- Tipe Template Literal
- Overload fungsi
- Tipe Rekursif
- Tipe Kondisional Rekursif
- Dukungan Modul ECMAScript di Node
- Fungsi Asersi
- Tipe Tuple Variadik
- Tipe Boxed
- Kovarians dan Kontravarians dalam TypeScript
    - Anotasi Varians Opsional untuk Parameter Tipe
- Index Signature Pola Template String
- Operator satisfies
- Import dan Export Khusus Tipe

    - Deklarasi using dan Pengelolaan Sumber Daya Eksplisit
        - Deklarasi await using
    - Atribut Import
    - Pemeriksaan Sintaks Ekspresi Reguler
    - import defer

Pendahuluan

Selamat datang di The Concise TypeScript Book! Panduan ini membekali Anda dengan pengetahuan penting dan keterampilan praktis untuk pengembangan TypeScript yang efektif. Temukan konsep dan teknik utama untuk menulis kode yang bersih dan tangguh. Baik Anda seorang pemula maupun pengembang berpengalaman, buku ini berfungsi sebagai panduan menyeluruh sekaligus referensi praktis untuk memanfaatkan kemampuan TypeScript dalam proyek Anda.

Buku ini membahas TypeScript 7.0.

Tentang Penulis

Simone Poggiali adalah Staff Engineer berpengalaman yang memiliki minat besar dalam menulis kode berkualitas profesional sejak era 90-an. Sepanjang karier internasionalnya, ia telah berkontribusi pada banyak proyek untuk beragam klien, mulai dari perusahaan rintisan hingga organisasi besar. Perusahaan ternama seperti HelloFresh, Siemens, O2, Leroy Merlin, dan Snowplow telah memperoleh manfaat dari keahlian dan dedikasinya.

Anda dapat menghubungi Simone Poggiali melalui platform berikut:

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- Email: gibbok.coding gmail.com

Daftar lengkap kontributor: https://github.com/gibbok/typescript-book/graphs/contributors

Pengenalan TypeScript

Apa itu TypeScript?

TypeScript adalah bahasa pemrograman bertipe kuat yang dikembangkan berdasarkan JavaScript. Bahasa ini awalnya dirancang oleh Anders Hejlsberg pada tahun 2012 dan saat ini dikembangkan serta dipelihara oleh Microsoft sebagai proyek sumber terbuka.

TypeScript dikompilasi menjadi JavaScript dan dapat dijalankan dalam lingkungan runtime JavaScript apa pun (misalnya, browser atau Node.js pada server).

TypeScript mendukung berbagai paradigma pemrograman seperti pemrograman fungsional, generic, imperatif, dan berorientasi objek, serta merupakan bahasa terkompilasi (ditranspilasi) yang dikonversi menjadi JavaScript sebelum dijalankan.

Mengapa TypeScript?

TypeScript adalah bahasa bertipe kuat yang membantu mencegah kesalahan pemrograman umum dan menghindari jenis error runtime tertentu sebelum program dijalankan.

Bahasa bertipe kuat memungkinkan pengembang menentukan berbagai batasan dan perilaku program dalam definisi tipe data, sehingga memudahkan verifikasi kebenaran perangkat lunak dan pencegahan cacat. Hal ini sangat bermanfaat dalam aplikasi berskala besar.

Beberapa manfaat TypeScript:

- Pengetikan statis, secara opsional bertipe kuat
- Inferensi Tipe
- Akses ke fitur ES6 dan ES7
- Kompatibilitas Lintas Platform dan Lintas Browser
- Dukungan tooling dengan IntelliSense

TypeScript dan JavaScript

TypeScript ditulis dalam berkas berekstensi `.ts` atau `.tsx`, sedangkan JavaScript ditulis dalam berkas berekstensi `.js` atau `.jsx`.

Berkas dengan ekstensi `.tsx` atau `.jsx` dapat berisi sintaks JSX untuk JavaScript, yang digunakan dalam React untuk pengembangan UI.

TypeScript adalah superset bertipe dari JavaScript (ECMAScript 2015) dalam hal sintaks. Semua kode JavaScript merupakan kode TypeScript yang valid, tetapi kebalikannya tidak selalu berlaku.

Sebagai contoh, perhatikan sebuah fungsi dalam berkas JavaScript dengan ekstensi `.js`, seperti berikut:

```
const sum = (a, b) => a + b;
```

Fungsi tersebut dapat dikonversi dan digunakan dalam TypeScript dengan mengubah ekstensi berkas menjadi `.ts`. Namun, jika fungsi yang sama diberi anotasi tipe TypeScript, fungsi tersebut tidak dapat dijalankan dalam lingkungan runtime JavaScript apa pun tanpa kompilasi. Kode TypeScript berikut akan menghasilkan error sintaks jika tidak dikompilasi:

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript dirancang untuk mendeteksi kemungkinan error runtime pada waktu kompilasi dengan memungkinkan pengembang mengekspresikan maksud melalui anotasi tipe. Selain itu, TypeScript juga dapat menemukan masalah tertentu meskipun tidak ada anotasi tipe eksplisit yang diberikan, berkat inferensi tipe. Sebagai contoh, cuplikan kode berikut tidak menentukan tipe TypeScript apa pun:

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

Dalam kasus ini, TypeScript mendeteksi error dan melaporkan:

```
Property 'y' does not exist on type '{ x: number; }'.
```

Sistem tipe TypeScript sangat dipengaruhi oleh perilaku runtime JavaScript. Sebagai contoh, operator penjumlahan (+), yang dalam JavaScript dapat melakukan penggabungan string atau penjumlahan numerik, dimodelkan dengan cara yang sama dalam TypeScript:

```
const result = '1' + 1; // Result is of type string
```

Tim di balik TypeScript telah mengambil keputusan yang disengaja untuk menandai penggunaan JavaScript yang tidak biasa sebagai error. Sebagai contoh, perhatikan kode JavaScript valid berikut:

```
const result = 1 + true; // In JavaScript, the result is equal to 2
```

Namun, TypeScript menghasilkan error:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Error ini terjadi karena TypeScript memberlakukan kompatibilitas tipe secara ketat, dan dalam kasus ini, TypeScript mengidentifikasi operasi yang tidak valid antara `number` dan `boolean`.

Pembuatan Kode TypeScript

Compiler TypeScript memiliki dua tanggung jawab utama: memeriksa error tipe dan mengompilasi menjadi JavaScript. Kedua proses ini tidak bergantung satu sama lain. Tipe tidak memengaruhi eksekusi kode dalam lingkungan runtime JavaScript karena tipe dihapus sepenuhnya selama kompilasi. TypeScript tetap dapat menghasilkan JavaScript meskipun terdapat error tipe.

Berikut adalah contoh kode TypeScript dengan error tipe:

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Argument of type 'string' is not assignable to parameter of type 'number'.
```

Namun, kode tersebut tetap dapat menghasilkan keluaran JavaScript yang dapat dijalankan:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

Tipe TypeScript tidak dapat diperiksa pada saat runtime. Sebagai contoh:

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // 'Dog' only refers to a type, but is being used as a
value here.
        // ...
    }
};
```

Karena tipe dihapus setelah kompilasi, kode ini tidak dapat dijalankan dalam JavaScript. Untuk mengenali tipe pada saat runtime, kita perlu menggunakan mekanisme lain. TypeScript menyediakan beberapa opsi, salah satu yang umum adalah “tagged union”. Sebagai contoh:

```
interface Dog {
    kind: 'dog'; // Tagged union
    bark: () => void;
}
```

```
interface Cat {
    kind: 'cat'; // Tagged union
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

Properti “kind” adalah nilai yang dapat digunakan pada saat runtime untuk membedakan objek dalam JavaScript.

Sebuah nilai pada saat runtime juga dapat memiliki tipe yang berbeda dari tipe yang dinyatakan dalam deklarasi tipe. Misalnya, pengembang dapat salah menafsirkan tipe API dan memberikan anotasi yang keliru.

TypeScript adalah superset dari JavaScript, sehingga kata kunci “class” dapat digunakan sebagai tipe dan nilai pada saat runtime.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
```

```
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

Dalam JavaScript, sebuah “class” memiliki properti “prototype”, dan operator “instanceof” dapat digunakan untuk menguji apakah properti prototype dari sebuah constructor muncul di mana pun dalam rantai prototype suatu objek.

TypeScript tidak berpengaruh pada performa runtime karena semua tipe akan dihapus. Namun, TypeScript menimbulkan sedikit overhead pada waktu build.

JavaScript Modern Saat Ini (Downleveling)

TypeScript dapat mengompilasi kode ke versi JavaScript apa pun yang telah dirilis sejak ECMAScript 3 (1999). Ini berarti TypeScript dapat mentranspilasi kode yang menggunakan fitur JavaScript terbaru ke versi yang lebih lama, sebuah proses yang dikenal sebagai Downleveling. Hal ini memungkinkan penggunaan JavaScript modern sekaligus mempertahankan kompatibilitas maksimal dengan lingkungan runtime yang lebih lama.

Penting untuk diperhatikan bahwa selama transpilasi ke versi JavaScript yang lebih lama, TypeScript mungkin menghasilkan kode yang dapat menimbulkan overhead performa dibandingkan dengan implementasi native.

Berikut adalah beberapa fitur JavaScript modern yang dapat digunakan dalam TypeScript:

- Modul ECMAScript sebagai pengganti callback “define” bergaya AMD atau pernyataan “require” CommonJS.
- Class sebagai pengganti prototype.
- Deklarasi variabel menggunakan “let” atau “const” sebagai pengganti “var”.
- Perulangan “for-of” atau “.forEach” sebagai pengganti perulangan “for” tradisional.
- Arrow function sebagai pengganti function expression.
- Destructuring assignment.
- Nama properti/metode yang disingkat dan nama properti terkomputasi.
- Parameter fungsi default.

Dengan memanfaatkan fitur JavaScript modern ini, pengembang dapat menulis kode TypeScript yang lebih ekspresif dan ringkas.

Memulai dengan TypeScript

Instalasi

Visual Studio Code menyediakan dukungan yang sangat baik untuk bahasa TypeScript, tetapi tidak menyertakan compiler TypeScript. Untuk menginstal compiler TypeScript, Anda dapat menggunakan package manager seperti npm atau yarn:

```
npm install typescript --save-dev
```

atau

```
yarn add typescript --dev
```

Pastikan untuk melakukan commit pada lockfile yang dihasilkan guna memastikan bahwa setiap anggota tim menggunakan versi TypeScript yang sama.

Untuk menjalankan compiler TypeScript, Anda dapat menggunakan perintah berikut:

```
npx tsc
```

atau

```
yarn tsc
```

Disarankan untuk menginstal TypeScript per proyek, bukan secara global, karena hal ini memberikan proses build yang lebih mudah diprediksi. Namun, untuk penggunaan sesekali, Anda dapat menggunakan perintah berikut:

```
npx tsc
```

atau menginstalnya secara global:

```
npm install -g typescript
```

Jika Anda menggunakan Microsoft Visual Studio, Anda dapat memperoleh TypeScript sebagai paket di NuGet untuk proyek MSBuild Anda. Di NuGet Package Manager Console, jalankan perintah berikut:

```
Install-Package Microsoft.TypeScript.MSBuild
```

Selama instalasi TypeScript, dua executable akan diinstal: “tsc” sebagai compiler TypeScript dan “tsserver” sebagai server mandiri TypeScript. Server mandiri tersebut berisi compiler dan layanan bahasa yang dapat dimanfaatkan oleh editor dan IDE untuk menyediakan pelengkapan kode cerdas.

Selain itu, tersedia beberapa transpiler yang kompatibel dengan TypeScript, seperti Babel (melalui plugin) atau swc. Transpiler ini dapat digunakan untuk mengonversi kode TypeScript ke bahasa atau versi target lainnya.

TypeScript 7.0 ditulis ulang dalam Go sebagai implementasi native dari compiler dan layanan bahasa. TypeScript menggunakan multithreading dengan memori bersama dan pengoptimalan lain untuk mempercepat build penuh serta fitur editor, sehingga mengurangi waktu tunggu umpan balik selama pengembangan.

Beberapa fitur performa TypeScript 7.0 dapat disesuaikan. Pemeriksaan tipe dapat berjalan dalam worker paralel dengan `--checkers`; lebih banyak worker dapat mempercepat proyek besar, tetapi menggunakan lebih banyak memori. Mode `--watch` yang dibangun ulang meningkatkan pemantauan berkas lintas

platform. TypeScript 7.0 belum menyertakan compiler API (per Juli 2026), sehingga tool yang masih memerlukan API TypeScript 6.0 dapat berjalan berdampingan dengan TypeScript 7.0 menggunakan `@typescript/typescript6` atau alias npm.

Konfigurasi

TypeScript dapat dikonfigurasi menggunakan opsi CLI `tsc` atau dengan memanfaatkan berkas konfigurasi khusus bernama `tsconfig.json` yang ditempatkan di root proyek.

Untuk menghasilkan berkas `tsconfig.json` yang telah diisi sebelumnya dengan pengaturan yang direkomendasikan, Anda dapat menggunakan perintah berikut:

```
tsc --init
```

Saat menjalankan perintah `tsc` secara lokal, TypeScript akan mengompilasi kode menggunakan konfigurasi yang ditentukan dalam berkas `tsconfig.json` terdekat.

Berikut adalah beberapa contoh perintah CLI yang dijalankan dengan pengaturan default:

```
tsc main.ts // Compile a specific file (main.ts) to JavaScript
tsc src/*.ts // Compile any .ts files under the 'src' folder to JavaScript
tsc app.ts util.ts --outfile index.js // Compile two TypeScript files (app.ts and util.ts) into a single JavaScript file (index.js)
```

Berkas Konfigurasi TypeScript

Berkas `tsconfig.json` digunakan untuk mengonfigurasi Compiler TypeScript (`tsc`). Biasanya, berkas ini ditambahkan ke

root proyek bersama dengan berkas `package.json`.

Catatan:

- `tsconfig.json` menerima komentar meskipun menggunakan format JSON.
- Sebaiknya gunakan berkas konfigurasi ini sebagai pengganti opsi baris perintah.

Pada tautan berikut, Anda dapat menemukan dokumentasi lengkap beserta skemanya:

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

Berikut adalah daftar konfigurasi yang umum dan berguna:

target

Properti "target" digunakan untuk menentukan versi ECMAScript yang menjadi target keluaran hasil kompilasi kode TypeScript Anda. Untuk browser modern, ES6 merupakan pilihan yang baik. Catatan: Dukungan ES5 dihentikan secara bertahap pada TypeScript 6.0 dan tidak lagi didukung pada TypeScript 7.0.

lib

Properti "lib" digunakan untuk menentukan berkas pustaka yang akan disertakan pada waktu kompilasi. TypeScript secara otomatis menyertakan API untuk fitur yang ditentukan dalam properti "target", tetapi pustaka tertentu dapat dihilangkan

atau dipilih untuk kebutuhan khusus. Misalnya, jika Anda mengerjakan proyek server, Anda dapat mengecualikan pustaka “DOM”, yang hanya berguna dalam lingkungan browser.

strict

Opsi “strict” meningkatkan keamanan tipe dengan mengaktifkan pemeriksaan yang lebih ketat. Opsi ini diaktifkan secara default mulai TypeScript 6.0; jika tidak, Anda harus secara eksplisit mengaturnya menjadi `true` dalam `tsconfig.json`. Mengaktifkan “strict” memungkinkan TypeScript untuk:

- Menghasilkan kode menggunakan “use strict” untuk setiap berkas sumber.
- Mempertimbangkan “null” dan “undefined” dalam proses pemeriksaan tipe.
- Menonaktifkan penggunaan tipe “any” ketika tidak ada anotasi tipe.
- Menghasilkan error saat ekspresi “this” digunakan, yang jika tidak, akan menyiratkan tipe “any”.

module

Properti “module” menetapkan sistem modul yang didukung untuk program hasil kompilasi. Pada saat runtime, module loader digunakan untuk menemukan dan menjalankan dependensi berdasarkan sistem modul yang ditentukan.

Module loader yang paling umum digunakan dalam JavaScript adalah CommonJS Node.js untuk aplikasi sisi server dan RequireJS untuk modul AMD dalam aplikasi web berbasis

browser. TypeScript dapat menghasilkan kode untuk berbagai sistem modul, termasuk UMD, System, ESNext, ES2015/ES6, dan ES2020. Sistem modul harus dipilih berdasarkan lingkungan target dan mekanisme pemuatan modul yang tersedia dalam lingkungan tersebut.

Catatan: Dukungan untuk sistem modul lama (AMD, UMD, SystemJS) dihentikan secara bertahap pada TypeScript 6.0 dan tidak lagi didukung pada TypeScript 7.0.

moduleResolution

Properti “moduleResolution” menentukan strategi resolusi modul. Gunakan “nodenext” atau “bundler” untuk kode TypeScript modern. Strategi “classic” hanya digunakan untuk versi lama TypeScript (sebelum 1.6).

esModuleInterop

Properti “esModuleInterop” memungkinkan import default dari modul CommonJS yang tidak melakukan export menggunakan properti “default”; properti ini menyediakan shim untuk memastikan kompatibilitas dalam JavaScript yang dihasilkan. Setelah mengaktifkan opsi ini, kita dapat menggunakan `import MyLibrary from "my-library"` sebagai pengganti `import * as MyLibrary from "my-library"`.

“esModuleInterop” pada awalnya bersifat opsional untuk menghindari breaking change, tetapi sejak lama telah menjadi default yang direkomendasikan. Menonaktifkannya dapat menyebabkan masalah runtime yang tidak kentara saat menggunakan CommonJS dengan ESM. Catatan: Mulai

TypeScript 6.0, perilaku interoperabilitas yang lebih aman ini selalu diaktifkan.

Pada TypeScript 6.0, beberapa opsi konfigurasi dan bentuk sintaks lama dihentikan secara bertahap atau tetap menggunakan perilaku lama selama masa transisi. Pada TypeScript 7.0, opsi dan bentuk tersebut menjadi hard error atau perilaku tanpa operasi.

Opsi dan bentuk yang dihentikan secara bertahap serta telah berubah menjadi hard error atau perilaku tanpa operasi adalah:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- menonaktifkan `esModuleInterop` atau `allowSyntheticDefaultImports`
- menonaktifkan `alwaysStrict`
- kata kunci `module` dalam deklarasi namespace
- `asserts` pada impor

- `/// <reference no-default-lib />` di bawah `skipDefaultLibCheck`
- jalur berkas CLI dengan `tsconfig.json` lokal, kecuali jika `--ignoreConfig` digunakan

jsx

Properti “jsx” hanya berlaku untuk berkas `.tsx` yang digunakan dalam ReactJS dan mengendalikan cara konstruksi JSX dikompilasi menjadi JavaScript. Opsi yang umum adalah “preserve”, yang akan mengompilasi menjadi berkas `.jsx` dengan mempertahankan JSX tanpa perubahan sehingga dapat diteruskan ke alat lain seperti Babel untuk transformasi lebih lanjut.

skipLibCheck

Properti “skipLibCheck” akan mencegah TypeScript melakukan pemeriksaan tipe terhadap keseluruhan paket pihak ketiga yang diimpor. Properti ini akan mengurangi waktu kompilasi suatu proyek. TypeScript tetap akan memeriksa kode Anda berdasarkan definisi tipe yang disediakan oleh paket-paket tersebut.

files

Properti “files” menunjukkan kepada compiler daftar berkas yang harus selalu disertakan dalam program.

include

Properti “include” menunjukkan kepada compiler daftar berkas yang ingin kita sertakan. Properti ini memungkinkan pola seperti glob, misalnya “*” *untuk subdirektori apa pun,* “” untuk nama berkas apa pun, dan “?” untuk karakter opsional.

exclude

Properti “exclude” menunjukkan kepada compiler daftar berkas yang tidak boleh disertakan dalam kompilasi. Ini dapat mencakup berkas seperti “node_modules” atau berkas pengujian. Catatan: `tsconfig.json` mengizinkan komentar.

importHelpers

TypeScript menggunakan kode bantuan saat menghasilkan kode untuk fitur JavaScript tingkat lanjut tertentu atau fitur yang diturunkan ke versi yang lebih lama. Secara default, kode bantuan ini diduplikasi dalam berkas yang menggunakannya. Sebagai gantinya, opsi `importHelpers` mengimpor kode bantuan tersebut dari modul `tslib`, sehingga keluaran JavaScript menjadi lebih efisien.

Saran Migrasi ke TypeScript

Untuk proyek besar, disarankan untuk menerapkan transisi bertahap, dengan kode TypeScript dan JavaScript pada awalnya digunakan secara berdampingan. Hanya proyek kecil yang dapat dimigrasikan ke TypeScript sekaligus.

Langkah pertama dalam transisi ini adalah memperkenalkan TypeScript ke dalam proses rantai build. Hal ini dapat dilakukan dengan menggunakan opsi compiler “allowJs”, yang

memungkinkan berkas `.ts` dan `.tsx` digunakan secara berdampingan dengan berkas JavaScript yang sudah ada. Karena TypeScript akan menggunakan tipe “any” sebagai pilihan terakhir untuk suatu variabel ketika tidak dapat menyimpulkan tipenya dari berkas JavaScript, disarankan untuk menonaktifkan “noImplicitAny” dalam opsi compiler Anda pada awal migrasi.

Langkah kedua adalah memastikan bahwa pengujian JavaScript Anda dapat berjalan bersama berkas TypeScript sehingga Anda dapat menjalankan pengujian saat mengonversi setiap modul. Jika Anda menggunakan Jest, pertimbangkan untuk menggunakan `ts-jest`, yang memungkinkan Anda menguji proyek TypeScript dengan Jest.

Langkah ketiga adalah menyertakan deklarasi tipe untuk pustaka pihak ketiga dalam proyek Anda. Deklarasi ini dapat ditemukan dalam paket pustaka atau di DefinitelyTyped. Anda dapat mencarinya melalui https://www.typescriptlang.org/dt/search dan menginstalnya dengan:

```
npm install --save-dev @types/package-name
```

atau

```
yarn add --dev @types/package-name
```

Langkah keempat adalah melakukan migrasi modul demi modul dengan pendekatan dari bawah ke atas, mengikuti grafik dependensi Anda yang dimulai dari simpul daun. Tujuannya adalah memulai dengan mengonversi modul yang tidak

bergantung pada modul lain. Untuk memvisualisasikan grafik dependensi, Anda dapat menggunakan alat “madge”.

Modul yang cocok untuk konversi awal ini adalah fungsi utilitas dan kode yang terkait dengan API atau spesifikasi eksternal. Definisi tipe TypeScript dapat dibuat secara otomatis dari kontrak Swagger, skema GraphQL, atau skema JSON untuk disertakan dalam proyek Anda.

Jika tidak tersedia spesifikasi atau skema resmi, Anda dapat membuat tipe dari data mentah, seperti JSON yang dikembalikan oleh server. Namun, disarankan untuk membuat tipe dari spesifikasi, bukan dari data, agar kasus khusus tidak terlewat.

Selama migrasi, hindari refactoring kode dan berfokuslah hanya pada penambahan tipe ke modul Anda.

Langkah kelima adalah mengaktifkan “noImplicitAny”, yang akan memastikan bahwa semua tipe diketahui dan didefinisikan, sehingga memberikan pengalaman penggunaan TypeScript yang lebih baik bagi proyek Anda.

Selama migrasi, Anda dapat menggunakan direktif `@ts-check`, yang mengaktifkan pemeriksaan tipe TypeScript dalam berkas JavaScript. Direktif ini menyediakan pemeriksaan tipe yang lebih longgar dan pada tahap awal dapat digunakan untuk mengidentifikasi masalah dalam berkas JavaScript. Ketika `@ts-check` disertakan dalam suatu berkas, TypeScript akan mencoba menyimpulkan definisi menggunakan komentar bergaya JSDoc. Namun, pertimbangkan untuk menggunakan anotasi JSDoc hanya pada tahap yang sangat awal dalam migrasi.

Pertimbangkan untuk mempertahankan nilai default `noEmitOnError` dalam `tsconfig.json` sebagai `false`. Ini akan memungkinkan Anda menghasilkan kode sumber JavaScript meskipun terdapat error yang dilaporkan.

Menjelajahi Sistem Tipe

Layanan Bahasa TypeScript

TypeScript Language Service, yang juga dikenal sebagai tsserver, menawarkan berbagai fitur seperti pelaporan error, diagnostik, kompilasi saat menyimpan, penggantian nama, menuju definisi, daftar pelengkapan, bantuan signature, dan lainnya. Layanan ini terutama digunakan oleh lingkungan pengembangan terintegrasi (IDE) untuk menyediakan dukungan IntelliSense. Layanan ini terintegrasi dengan lancar dengan Visual Studio Code dan digunakan oleh alat seperti Conquer of Completion (Coc).

Pengembang dapat memanfaatkan API khusus dan membuat plugin language service kustom mereka sendiri untuk meningkatkan pengalaman penyuntingan TypeScript. Hal ini dapat sangat berguna untuk menerapkan fitur linting khusus atau mengaktifkan pelengkapan otomatis bagi bahasa template kustom.

Contoh plugin kustom yang digunakan di dunia nyata adalah “typescript-styled-plugin”, yang menyediakan pelaporan error sintaks dan dukungan IntelliSense untuk properti CSS dalam styled component.

Untuk informasi lebih lanjut dan panduan memulai cepat, Anda dapat merujuk ke Wiki TypeScript resmi di GitHub: https://github.com/microsoft/TypeScript/wiki/

Pengetikan Struktural

TypeScript didasarkan pada sistem tipe struktural. Artinya, kompatibilitas dan ekuivalensi tipe ditentukan oleh struktur atau definisi aktual tipe tersebut, bukan oleh nama atau tempat deklarasinya seperti dalam sistem tipe nominal, misalnya C# atau C.

Sistem tipe struktural TypeScript dirancang berdasarkan cara kerja sistem duck typing dinamis JavaScript saat runtime.

Contoh berikut merupakan kode TypeScript yang valid. Seperti yang dapat Anda amati, “X” dan “Y” memiliki member “a” yang sama meskipun nama deklarasinya berbeda. Tipe ditentukan oleh strukturnya, dan dalam kasus ini, karena strukturnya sama, keduanya kompatibel dan valid.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // Valid
```

Aturan Dasar Perbandingan TypeScript

Proses perbandingan TypeScript bersifat rekursif dan dijalankan pada tipe yang bersarang di tingkat mana pun.

Tipe “X” kompatibel dengan “Y” jika “Y” setidaknya memiliki member yang sama dengan “X”.

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // Valid, as it has at least the
same members as X
const r: X = y;
```

Parameter fungsi dibandingkan berdasarkan tipe, bukan namanya:

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Valid
x = y; // Valid
```

Tipe kembalian fungsi harus sama:

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Invalid
x = y; // Invalid
```

Tipe kembalian fungsi sumber harus merupakan subtipe dari tipe kembalian fungsi target:

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Valid
y = x; // Invalid member b is missing
```

Mengabaikan parameter fungsi diperbolehkan karena merupakan praktik umum dalam JavaScript, misalnya saat menggunakan “Array.prototype.map()”:

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Oleh karena itu, deklarasi tipe berikut sepenuhnya valid:

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Missing b parameter
y = x; // Valid
```

Setiap parameter opsional tambahan pada tipe sumber adalah valid:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; //Valid
```

Setiap parameter opsional pada tipe target yang tidak memiliki parameter terkait pada tipe sumber adalah valid dan bukan merupakan error:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; // Valid
```

Parameter rest diperlakukan sebagai rangkaian parameter opsional yang tidak terbatas:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //valid
```

Fungsi dengan overload valid jika signature overload kompatibel dengan signature implementasinya:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Valid
x('a', 1); // Valid

function y(a: string): void; // Invalid, not compatible with implementation signature
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
y('a');
y('a', 1);
```

Perbandingan parameter fungsi berhasil jika parameter sumber dan target dapat ditetapkan ke supertipe atau subtipe (bivariance).

```
// Supertype
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Subtype
class Y extends X {}
// Subtype
class Z extends X {}
```

```
type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// Bivariance does accept supertypes
console.log(getA(new X('x'))); // Valid
console.log(getA(new Y('Y'))); // Valid
console.log(getA(new Z('z'))); // Valid
```

Enum dapat dibandingkan dan kompatibel dengan angka, demikian pula sebaliknya, tetapi membandingkan nilai Enum dari tipe Enum yang berbeda adalah tidak valid.

```
enum X {
    A,
    B,
}
enum Y {
    A,
    B,
    C,
}
const xa: number = X.A; // Valid
const ya: Y = 0; // Valid
X.A === Y.A; // Invalid
```

Instance dari suatu class menjalani pemeriksaan kompatibilitas untuk member private dan protected:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
```

```
    constructor(value: string) {
        this.a = value;
    }
}

let x: X = new Y('y'); // Invalid
```

Pemeriksaan perbandingan tidak mempertimbangkan perbedaan hierarki pewarisan, misalnya:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Valid
x === z; // Valid even if z is from a different inheritance hierarchy
```

Tipe generic dibandingkan berdasarkan strukturnya setelah parameter generic diterapkan; hanya tipe hasil akhir yang

dibandingkan sebagai tipe non-generic.

```
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Invalid as the type argument is used in the final
structure

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Valid as the type argument is not used in the final
structure
```

Ketika argumen tipe pada generic tidak ditentukan, semua argumen yang tidak ditentukan diperlakukan sebagai tipe “any”:

```
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Valid
```

Ingat:

```
let a: number = 1;
let b: number = 2;
a = b; // Valid, everything is assignable to itself

let c: any;
c = 1; // Valid, all types are assignable to any

let d: unknown;
d = 1; // Valid, all types are assignable to unknown
```

```
let e: unknown;
let e1: unknown = e; // Valid, unknown is only assignable to
itself and any
let e2: any = e; // Valid
let e3: number = e; // Invalid

let f: never;
f = 1; // Invalid, nothing is assignable to never

let g: void;
let g1: any;
g = 1; // Invalid, void is not assignable to or from anything
except any
g = g1; // Valid
```

Harap perhatikan bahwa ketika “strictNullChecks” diaktifkan, “null” dan “undefined” diperlakukan serupa dengan “void”; jika tidak, keduanya serupa dengan “never”.

Tipe sebagai Himpunan

Dalam TypeScript, tipe adalah himpunan nilai yang mungkin. Himpunan ini juga disebut sebagai domain tipe. Setiap nilai dari suatu tipe dapat dipandang sebagai elemen dalam sebuah himpunan. Tipe menetapkan batasan yang harus dipenuhi oleh setiap elemen dalam himpunan agar dianggap sebagai anggota himpunan tersebut. Tugas utama TypeScript adalah memeriksa dan memverifikasi apakah satu himpunan merupakan subhimpunan dari himpunan lain.

TypeScript mendukung berbagai jenis himpunan:

Istilah himpunan	TypeScript	Catatan
Himpunan kosong	never	“never” berisi apa pun selain dirinya sendiri

Himpunan satu elemen	undefined / null / literal type	
Himpunan berhingga	boolean / union	
Himpunan tak berhingga	string / number / object	
Himpunan semesta	any / unknown	Setiap elemen merupakan anggota “any” dan setiap himpunan merupakan subhimpunannya / “unknown” adalah padanan “any” yang aman terhadap tipe

Berikut beberapa contoh:

TypeScript	**Istilah himpunan**	**Contoh**
never	∅ (himpunan kosong)	const x: never = ‘x’; // Error: Type ‘string’ is not assignable to type ‘never’
Tipe literal	Himpunan satu elemen	type X = ‘X’;
		type Y = 7;
Nilai dapat ditetapkan ke T	Nilai ∈ T (anggota dari)	type XY = ‘X’ | ‘Y’;
		const x: XY = ‘X’;
T1 dapat ditetapkan ke T2	T1 ⊆ T2 (subhimpunan dari)	type XY = ‘X’ | ‘Y’;
		const x: XY = ‘X’;
		const j: XY = ‘J’; // Type ‘“J”’ is not assignable to type ‘XY’.
T1 extends T2	T1 ⊆ T2 (subhimpunan dari)	type X = ‘X’ extends string ? true : false;
T1 | T2	T1 ∪ T2 (union)	type XY = ‘X’ | ‘Y’;

		type JK = 1 | 2;
T1 & T2	T1 ∩ T2 (intersection)	type X = { a: string }
		type Y = { b: string }
		type XY = X & Y
		const x: XY = { a: ‘a’, b: ‘b’ }
unknown	Himpunan semesta	const x: unknown = 1

Union, (T1 | T2), menghasilkan himpunan yang lebih luas (keduanya):

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Valid
```

Intersection, (T1 & T2), menghasilkan himpunan yang lebih sempit (hanya yang sama):

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Invalid
const j: XY = { a: 'a', b: 'b' }; // Valid
```

Kata kunci `extends` dapat dianggap sebagai “subhimpunan dari” dalam konteks ini. Kata kunci tersebut menetapkan batasan untuk suatu tipe. Ketika `extends` digunakan dengan generic, kata kunci tersebut membatasi parameter tipe generic ke tipe yang lebih spesifik.

Harap perhatikan bahwa `extends` di sini tidak berkaitan dengan pewarisan class dalam pengertian OOP.

TypeScript bekerja dengan tipe struktural dan tidak memiliki hierarki nominal yang ketat. Bahkan, seperti pada contoh di bawah, dua tipe dapat saling tumpang-tindih tanpa salah satunya menjadi subtipe dari yang lain karena TypeScript mempertimbangkan struktur, atau bentuk, objek.

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
```

```
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Valid
```

Menetapkan Tipe: Deklarasi Tipe dan Asersi Tipe

Tipe dapat ditetapkan dengan berbagai cara dalam TypeScript:

Deklarasi Tipe

Dalam contoh berikut, kita menggunakan x: X (“: Tipe”) untuk mendeklarasikan tipe bagi variabel x.

```
type X = {
    a: string;
};

// Type declaration
const x: X = {
    a: 'a',
};
```

Jika variabel tidak memiliki format yang ditentukan, TypeScript akan melaporkan error. Misalnya:

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
     b: 'b', // Error: Object literal may only specify known
properties
};
```

Asersi Tipe

Asersi dapat ditambahkan menggunakan kata kunci `as`. Ini memberi tahu compiler bahwa pengembang memiliki lebih banyak informasi tentang suatu tipe dan meniadakan error apa pun yang mungkin terjadi.

Sebagai contoh:

```
type X = {
    a: string;
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

Dalam contoh di atas, objek x diasersikan memiliki tipe X menggunakan kata kunci as. Ini memberi tahu compiler TypeScript bahwa objek tersebut sesuai dengan tipe yang ditentukan meskipun memiliki properti tambahan b yang tidak terdapat dalam definisi tipe.

Asersi tipe berguna dalam situasi ketika tipe yang lebih spesifik perlu ditentukan, terutama saat bekerja dengan DOM. Misalnya:

```
const myInput = document.getElementById('my_input') as HTMLInputElement;
```

Di sini, asersi tipe as HTMLInputElement digunakan untuk memberi tahu TypeScript bahwa hasil getElementById harus diperlakukan sebagai HTMLInputElement. Asersi tipe juga dapat digunakan untuk memetakan ulang kunci, seperti yang ditunjukkan dalam contoh di bawah dengan template literal:

```
type J<Type> = {
    [
        Property in keyof Type as `prefix_${string & Property}`
    ]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

Dalam contoh ini, tipe `J<Type>` menggunakan mapped type dengan template literal untuk memetakan ulang kunci dari Type. Tipe tersebut membuat properti baru dengan "prefix_" yang ditambahkan ke setiap kunci, dan nilai terkaitnya adalah fungsi yang mengembalikan nilai properti asli.

Perlu diperhatikan bahwa saat menggunakan asersi tipe, TypeScript tidak akan menjalankan pemeriksaan properti berlebih. Oleh karena itu, umumnya lebih baik menggunakan deklarasi tipe ketika struktur objek telah diketahui sebelumnya.

Deklarasi Ambient

Deklarasi ambient adalah berkas yang mendeskripsikan tipe untuk kode JavaScript; berkas tersebut memiliki format nama berkas `.d.ts`. Deklarasi ini biasanya diimpor dan digunakan untuk memberikan anotasi pada pustaka JavaScript yang sudah ada atau untuk menambahkan tipe ke berkas JS yang sudah ada dalam proyek Anda.

Tipe untuk banyak pustaka umum dapat ditemukan di: https://github.com/DefinitelyTyped/DefinitelyTyped/

dan dapat diinstal menggunakan:

```
npm install --save-dev @types/library-name
```

Untuk Deklarasi Ambient yang Anda definisikan, Anda dapat mengimpornya menggunakan referensi “triple-slash”:

```
/// <reference path="./library-types.d.ts" />
```

Anda dapat menggunakan Deklarasi Ambient bahkan di dalam file JavaScript dengan menggunakan `// @ts-check`.

Kata kunci `declare` memungkinkan definisi tipe untuk kode JavaScript yang sudah ada tanpa mengimpornya, serta berfungsi sebagai placeholder untuk tipe dari file lain atau tipe global.

Pemeriksaan Properti dan Pemeriksaan Properti Berlebih

TypeScript didasarkan pada sistem tipe struktural, tetapi pemeriksaan properti berlebih merupakan fitur TypeScript yang memungkinkannya memeriksa apakah suatu objek memiliki properti persis seperti yang ditentukan dalam tipe.

Pemeriksaan Properti Berlebih dilakukan ketika menetapkan objek literal ke variabel atau ketika meneruskannya sebagai argumen ke properti berlebih fungsi, misalnya.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
```

```
const x: X = y; // Valid because structural typing
const w: X = { a: 'a', b: 'b' }; // Invalid because excess
property checking
```

Tipe Lemah

Suatu tipe dianggap lemah jika hanya berisi sekumpulan properti yang semuanya opsional:

```
type X = {
    a?: string;
    b?: string;
};
```

TypeScript menganggap penetapan apa pun ke tipe lemah sebagai error jika tidak ada tumpang-tindih. Misalnya, kode berikut menghasilkan error:

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // Invalid
```

Meskipun tidak disarankan, jika diperlukan, pemeriksaan ini dapat dilewati dengan menggunakan asersi tipe:

```
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Valid
```

Atau dengan menambahkan `unknown` ke index signature pada tipe lemah:

```
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Valid
```

Pemeriksaan Ketat Object Literal (Freshness)

Pemeriksaan ketat objek literal, yang terkadang disebut sebagai “freshness”, adalah fitur dalam TypeScript yang membantu menemukan properti berlebih atau salah eja yang mungkin tidak terdeteksi dalam pemeriksaan tipe struktural biasa.

Saat membuat objek literal, compiler TypeScript menganggapnya “fresh”. Jika objek literal tersebut ditetapkan ke variabel atau diteruskan sebagai parameter, TypeScript akan menghasilkan error jika objek literal menentukan properti yang tidak ada dalam tipe target.

Namun, “freshness” hilang ketika objek literal diperlebar atau asersi tipe digunakan.

Berikut beberapa contoh untuk menggambarkannya:

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Freshness check: Invalid assignment
var y: Y;
```

```
y = { a: 'a', bx: 'bx' }; // Freshness check: Invalid assignment

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Widening: No errors, structurally type compatible

fn({ a: 'a', bx: 'b' }); // Freshness check: Invalid argument

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Widening: No Freshness check
```

Inferensi Tipe

TypeScript dapat menyimpulkan tipe ketika tidak ada anotasi yang diberikan dalam situasi berikut:

- Inisialisasi variabel.
- Inisialisasi member.
- Menetapkan nilai default untuk parameter.
- Tipe kembalian fungsi.

Sebagai contoh:

```
let x = 'x'; // The type inferred is string
```

Compiler TypeScript menganalisis nilai atau ekspresi dan menentukan tipenya berdasarkan informasi yang tersedia.

Inferensi Lebih Lanjut

Ketika beberapa ekspresi digunakan dalam inferensi tipe, TypeScript mencari “tipe umum terbaik”. Misalnya:

```
let x = [1, 'x', 1, null]; // The type inferred is: (string | number | null)[]
```

Jika compiler tidak dapat menemukan tipe umum terbaik, compiler mengembalikan union type. Sebagai contoh:

```
let x = [new RegExp('x'), new Date()]; // Type inferred is: (RegExp | Date)[]
```

TypeScript menggunakan "contextual typing" berdasarkan lokasi variabel untuk menyimpulkan tipe. Dalam contoh berikut, compiler mengetahui bahwa `e` bertipe `MouseEvent` karena tipe event `click` didefinisikan dalam berkas lib.d.ts, yang berisi deklarasi ambient untuk berbagai konstruksi umum JavaScript dan DOM:

```
window.addEventListener('click', function (e) {}); // The inferred type of e is MouseEvent
```

Pelebaran Tipe

Pelebaran tipe adalah proses ketika TypeScript menetapkan tipe ke variabel yang diinisialisasi tanpa anotasi tipe. Proses ini memungkinkan tipe sempit diperlebar, tetapi tidak sebaliknya. Dalam contoh berikut:

```
let x = 'x'; // TypeScript infers as string, a wide type
let y: 'y' | 'x' = 'y'; // y types is a union of literal types
y = x; // Invalid Type 'string' is not assignable to type '"x" | "y"'.
```

TypeScript menetapkan `string` ke `x` berdasarkan satu nilai yang diberikan saat inisialisasi (`x`); ini merupakan contoh pelebaran.

TypeScript menyediakan cara untuk mengendalikan proses pelebaran, misalnya dengan menggunakan "const".

Const

Menggunakan kata kunci `const` saat mendeklarasikan variabel menghasilkan inferensi tipe yang lebih sempit dalam TypeScript.

Sebagai contoh:

```
const x = 'x'; // TypeScript infers the type of x as 'x', a narrower type
let y: 'y' | 'x' = 'y';
y = x; // Valid: The type of x is inferred as 'x'
```

Dengan menggunakan `const` untuk mendeklarasikan variabel x, tipenya dipersempit ke nilai literal tertentu, yaitu ‘x’. Karena tipe x dipersempit, nilai tersebut dapat ditetapkan ke variabel y tanpa error. Tipe tersebut dapat disimpulkan karena variabel `const` tidak dapat ditetapkan ulang, sehingga tipenya dapat dipersempit ke tipe literal tertentu; dalam kasus ini, tipe literal ‘x’.

Modifier Const pada Parameter Tipe

Mulai TypeScript versi 5.0, atribut `const` dapat ditentukan pada parameter tipe generic. Ini memungkinkan inferensi tipe yang paling presisi. Mari kita lihat contoh tanpa menggunakan `const`:

```
function identity<T>(value: T) {
    // No const here
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred is: { a: string; b: string; }
```

Seperti yang dapat Anda lihat, properti `a` dan `b` disimpulkan dengan tipe `string`.

Sekarang, mari kita lihat perbedaannya dengan versi `const`:

```
function identity<const T>(value: T) {
    // Using const modifier on type parameters
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred is: { a: "a"; b: "b"; }
```

Sekarang kita dapat melihat bahwa properti `a` dan `b` disimpulkan sebagai string literal, bukan sekadar tipe `string`.

Asersi Const

Fitur ini memungkinkan Anda mendeklarasikan variabel dengan tipe literal yang lebih presisi berdasarkan nilai inisialisasinya, yang menandakan kepada compiler bahwa nilai tersebut harus diperlakukan sebagai literal yang immutable. Berikut beberapa contoh:

Pada satu properti:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

Pada keseluruhan objek:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

Hal ini dapat sangat berguna ketika mendefinisikan tipe untuk tuple:

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Tuple of readonly [1, 2, 3]
```

Anotasi Tipe Eksplisit

Kita dapat menentukan dan memberikan tipe secara eksplisit. Dalam contoh berikut, properti `x` bertipe `number`:

```
const v = {
    x: 1, // Inferred type: number (widening)
};
v.x = 3; // Valid
```

Kita dapat membuat anotasi tipe lebih spesifik dengan menggunakan union dari tipe literal:

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x is now a union of literal types: 1 | 2 | 3
};
v.x = 3; // Valid
v.x = 100; // Invalid
```

Penyempitan Tipe

Penyempitan tipe adalah proses dalam TypeScript ketika tipe umum dipersempit menjadi tipe yang lebih spesifik. Hal ini terjadi ketika TypeScript menganalisis kode dan menentukan bahwa kondisi atau operasi tertentu dapat memperjelas informasi tipe.

Penyempitan tipe dapat terjadi dengan berbagai cara, termasuk:

Kondisi

Dengan menggunakan pernyataan kondisional, seperti `if` atau `switch`, TypeScript dapat mempersempit tipe berdasarkan hasil kondisi. Sebagai contoh:

```
let x: number | undefined = 10;

if (x !== undefined) {
    x += 100; // The type is number, which had been narrowed by the condition
}
```

Melempar atau Mengembalikan

Melempar error atau mengembalikan nilai lebih awal dari sebuah branch dapat digunakan untuk membantu TypeScript mempersempit tipe. Sebagai contoh:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

Cara lain untuk mempersempit tipe dalam TypeScript meliputi:

- Operator `instanceof`: Digunakan untuk memeriksa apakah suatu objek merupakan instance dari class tertentu.
- Operator `in`: Digunakan untuk memeriksa apakah suatu properti terdapat dalam objek.

- Operator `typeof`: Digunakan untuk memeriksa tipe suatu nilai saat runtime.
- Fungsi bawaan seperti `Array.isArray()`: Digunakan untuk memeriksa apakah suatu nilai merupakan array.

Union Terdiskriminasi

"Union Terdiskriminasi" merupakan pola dalam TypeScript ketika sebuah "tag" eksplisit ditambahkan ke objek untuk membedakan berbagai tipe di dalam sebuah union. Pola ini juga disebut sebagai "tagged union". Dalam contoh berikut, "tag" direpresentasikan oleh properti "type":

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // type is A
        case 'type_b':
            return input.value + 'extra'; // type is B
    }
};
```

Type Guard Buatan Pengguna

Dalam kasus ketika TypeScript tidak dapat menentukan suatu tipe, kita dapat menulis fungsi pembantu yang dikenal sebagai "type guard buatan pengguna". Dalam contoh berikut, kita akan menggunakan predikat tipe untuk mempersempit tipe setelah menerapkan pemfilteran tertentu:

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // The type is (string
| null)[], TypeScript was not able to infer the type properly

const isValid = (item: string | null): item is string => item
!== null; // Custom type guard

const  r2  =  data.filter(isValid);  //  The  type  is  fine  now
string[], by using the predicate type guard we were able to
narrow the type
```

Penyempitan switch-true

TypeScript 5.3 menambahkan penyempitan switch-true, yang memungkinkan Anda mengganti rangkaian if/else yang rumit dengan switch (true) menggunakan kondisi boolean. Fitur ini meningkatkan keterbacaan dan tetap mempersempit tipe. Fitur ini serupa dengan pattern matching, tetapi lebih sederhana.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

Tipe Primitif

TypeScript mendukung 7 tipe primitif. Tipe data primitif merujuk pada tipe yang bukan merupakan objek dan tidak memiliki metode apa pun yang terkait dengannya. Dalam TypeScript, semua tipe primitif bersifat immutable, yang berarti nilainya tidak dapat diubah setelah ditetapkan.

string

Tipe primitif `string` menyimpan data tekstual, dan nilainya selalu diapit tanda kutip ganda atau tunggal.

```
const x: string = 'x';
const y: string = 'y';
```

String dapat mencakup beberapa baris jika diapit oleh karakter backtick (`):

```
let sentence: string = `xxx,
   yyy`;
```

boolean

Tipe data `boolean` dalam TypeScript menyimpan nilai biner, yaitu `true` atau `false`.

```
const isReady: boolean = true;
```

number

Tipe data `number` dalam TypeScript direpresentasikan dengan nilai floating-point 64-bit. Tipe `number` dapat merepresentasikan bilangan bulat dan pecahan. TypeScript juga mendukung bilangan heksadesimal, biner, dan oktal, misalnya:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Hexadecimal starts with
0x
const binary: number = 0b1010; // Binary starts with 0b
const octal: number = 0o633; // Octal starts with 0o
```

bigint

`bigint` merepresentasikan nilai bilangan bulat yang dapat lebih besar daripada bilangan bulat aman maksimum yang didukung oleh `number`, yaitu 2^53 - 1.

`bigint` dapat dibuat dengan memanggil fungsi bawaan `BigInt()` atau dengan menambahkan `n` di akhir literal numerik bilangan bulat apa pun:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Catatan:

- Nilai `bigint` tidak dapat dicampur dengan `number` dan tidak dapat digunakan dengan `Math` bawaan; nilai-nilai tersebut harus dikonversi ke tipe yang sama.
- Nilai `bigint` hanya tersedia jika konfigurasi target adalah ES2020 atau yang lebih tinggi.

Symbol

Symbol adalah pengidentifikasi unik yang dapat digunakan sebagai kunci properti dalam objek untuk mencegah konflik penamaan.

```
type Obj = {
    [sym: symbol]: number;
```

```
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null dan undefined

Tipe `null` dan `undefined` sama-sama merepresentasikan ketiadaan nilai atau tidak adanya nilai apa pun.

Tipe `undefined` berarti nilai belum ditetapkan atau diinisialisasi, atau menunjukkan ketiadaan nilai yang tidak disengaja.

Tipe `null` berarti kita mengetahui bahwa field tersebut tidak memiliki nilai, sehingga nilainya tidak tersedia, dan ini menunjukkan ketiadaan nilai yang disengaja.

Array

`array` adalah tipe data yang dapat menyimpan beberapa nilai dengan tipe yang sama atau berbeda. Array dapat didefinisikan menggunakan sintaks berikut:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Union
```

TypeScript mendukung array readonly menggunakan sintaks berikut:

```
const x: readonly string[] = ['a', 'b']; // Readonly modifier
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Invalid
```

TypeScript mendukung tuple dan tuple readonly:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

Tipe data any secara harfiah merepresentasikan "sembarang" nilai dan merupakan tipe default ketika TypeScript tidak dapat menginferensi tipe atau ketika tipe tidak ditentukan.

Saat menggunakan any, compiler TypeScript melewati pemeriksaan tipe, sehingga tidak ada keamanan tipe ketika any digunakan. Secara umum, jangan gunakan any untuk membungkam compiler saat terjadi error; sebaliknya, berfokuslah untuk memperbaiki error tersebut, karena penggunaan any memungkinkan kontrak dilanggar dan manfaat autocomplete TypeScript hilang.

Tipe any dapat berguna selama migrasi bertahap dari JavaScript ke TypeScript, karena tipe ini dapat membungkam compiler.

Untuk proyek baru, gunakan konfigurasi TypeScript noImplicitAny, yang memungkinkan TypeScript melaporkan error ketika any digunakan atau diinferensi.

Tipe any biasanya menjadi sumber error yang dapat menyamarkan masalah sebenarnya pada tipe Anda. Hindari menggunakannya sebisa mungkin.

Anotasi Tipe

Pada variabel yang dideklarasikan menggunakan `var`, `let`, dan `const`, anotasi tipe dapat ditambahkan secara opsional:

```
const x: number = 1;
```

TypeScript dapat menginferensi tipe dengan baik, terutama untuk tipe yang sederhana, sehingga deklarasi ini tidak diperlukan dalam sebagian besar kasus.

Pada fungsi, anotasi tipe dapat ditambahkan ke parameter:

```
function sum(a: number, b: number) {
    return a + b;
}
```

Berikut adalah contoh yang menggunakan fungsi anonim (juga disebut fungsi lambda):

```
const sum = (a: number, b: number) => a + b;
```

Anotasi ini dapat dihindari ketika terdapat nilai default untuk sebuah parameter:

```
const sum = (a = 10, b: number) => a + b;
```

Anotasi tipe kembalian dapat ditambahkan ke fungsi:

```
const sum = (a = 10, b: number): number => a + b;
```

Hal ini sangat berguna untuk fungsi yang lebih kompleks, karena menuliskan tipe kembalian sebelum implementasi dapat membantu Anda memikirkan fungsi tersebut secara menyeluruh.

Secara umum, pertimbangkan untuk memberi anotasi pada signature tipe, tetapi tidak pada variabel lokal di dalam isi fungsi, dan selalu tambahkan tipe pada literal objek.

Properti Opsional

Sebuah objek dapat menentukan properti opsional dengan menambahkan tanda tanya `?` di akhir nama properti:

```
type X = {
    a: number;
    b?: number; // Optional
};
```

Nilai default dapat ditentukan ketika sebuah properti bersifat opsional:

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

Properti Readonly

Penulisan ulang suatu properti dapat dicegah dengan menggunakan modifier `readonly`, yang memastikan bahwa properti tersebut tidak dapat ditulis ulang, tetapi tidak memberikan jaminan immutability secara menyeluruh:

```
interface Y {
    readonly a: number;
}

type X = {
```

```
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

Index Signature

Dalam TypeScript, kita dapat menggunakan `string`, `number`, dan `symbol` sebagai index signature:

```
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // Same result as k[1]
```

Perhatikan bahwa JavaScript secara otomatis mengonversi indeks dengan `number` menjadi indeks dengan `string`, sehingga `k[1]` atau `k["1"]` mengembalikan nilai yang sama.

Memperluas Tipe

Sebuah `interface` dapat diperluas (menyalin member dari tipe lain):

```
interface X {
    a: string;
}
```

```
interface Y extends X {
    b: string;
}
```

Interface juga dapat diperluas dari beberapa tipe:

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

Kata kunci `extends` hanya berfungsi pada interface dan class; untuk type, gunakan intersection:

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

Sebuah type dapat diperluas menggunakan interface, tetapi tidak sebaliknya:

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

Tipe Literal

Tipe Literal adalah himpunan berelemen tunggal di dalam tipe kolektif; tipe ini mendefinisikan nilai yang sangat spesifik dan merupakan primitif JavaScript.

Tipe Literal dalam TypeScript adalah number, string, dan boolean.

Contoh literal:

```
const a = 'a'; // String literal type
const b = 1; // Numeric literal type
const c = true; // Boolean literal type
```

Tipe literal string, numerik, dan boolean digunakan dalam union, type guard, dan type alias. Dalam contoh berikut, Anda dapat melihat sebuah type alias union. `O` hanya terdiri dari nilai-nilai yang ditentukan; string lain tidak valid:

```
type O = 'a' | 'b' | 'c';
```

Inferensi Literal

Inferensi Literal adalah fitur dalam TypeScript yang memungkinkan tipe suatu variabel atau parameter diinferensi berdasarkan nilainya.

Dalam contoh berikut, kita dapat melihat bahwa TypeScript menganggap `x` sebagai tipe literal karena nilainya tidak dapat diubah di kemudian hari, sedangkan `y` diinferensi sebagai string karena dapat diubah di kemudian hari.

```
const x = 'x'; // Literal type of 'x', because this value cannot be changed
let y = 'y'; // Type string, as we can change this value
```

Dalam contoh berikut, kita dapat melihat bahwa `o.x` diinferensi sebagai `string` (dan bukan literal `a`) karena TypeScript menganggap nilainya dapat diubah di kemudian hari.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // This is a wider string
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x));  // Argument of type 'string' is not
assignable to parameter of type 'X'
```

Seperti yang dapat Anda lihat, kode tersebut menghasilkan error saat meneruskan `o.x` ke `fn` karena `X` merupakan tipe yang lebih sempit.

Kita dapat mengatasi masalah ini dengan menggunakan type assertion dengan `const` atau tipe `X`:

```
let o = {
    x: 'a' as const,
};
```

atau:

```
let o = {
    x: 'a' as X,
};
```

strictNullChecks

`strictNullChecks` adalah opsi compiler TypeScript yang memberlakukan pemeriksaan null secara ketat. Ketika opsi ini

diaktifkan, variabel dan parameter hanya dapat diberi nilai `null` atau `undefined` jika telah dideklarasikan secara eksplisit sebagai tipe tersebut menggunakan tipe union `null | undefined`. Jika variabel atau parameter tidak dideklarasikan secara eksplisit sebagai nullable, TypeScript akan menghasilkan error untuk mencegah potensi error saat runtime.

Enum

Dalam TypeScript, `enum` adalah sekumpulan nilai konstanta bernama.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

Enum dapat didefinisikan dengan berbagai cara:

Enum numerik

Dalam TypeScript, Enum Numerik adalah Enum yang setiap konstantanya diberi nilai numerik, dimulai dari 0 secara default.

```
enum Size {
    Small, // value starts from 0
    Medium,
    Large,
}
```

Nilai kustom dapat ditentukan dengan menetapkannya secara eksplisit:

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

Enum string

Dalam TypeScript, Enum String adalah Enum yang setiap konstantanya diberi nilai string.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

Catatan: TypeScript memungkinkan penggunaan Enum heterogen ketika member string dan numerik dapat digunakan bersama.

Enum konstan

Enum konstan dalam TypeScript adalah jenis Enum khusus yang semua nilainya diketahui pada waktu kompilasi dan disisipkan secara inline di setiap tempat enum digunakan, sehingga menghasilkan kode yang lebih efisien.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

Akan dikompilasi menjadi:

```
console.log('EN' /* Language.English */);
```

Catatan: Const enum memiliki nilai hardcoded, yang menghapus enum, sehingga dapat lebih efisien dalam library mandiri tetapi secara umum tidak diinginkan. Selain itu, const enum tidak dapat memiliki member yang dihitung.

Pemetaan balik

Dalam TypeScript, pemetaan balik pada Enum mengacu pada kemampuan untuk mengambil nama anggota Enum dari nilainya. Secara default, anggota Enum memiliki pemetaan maju dari nama ke nilai, tetapi pemetaan balik dapat dibuat dengan menetapkan nilai secara eksplisit untuk setiap anggota. Pemetaan balik berguna ketika Anda perlu mencari anggota Enum berdasarkan nilainya, atau ketika Anda perlu mengiterasi semua anggota Enum. Perhatikan bahwa hanya anggota enum numerik yang akan menghasilkan pemetaan balik, sedangkan anggota enum string sama sekali tidak menghasilkan pemetaan balik.

Enum berikut:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

Dikompilasi menjadi:

```
'use strict';
var Grade;
(function (Grade) {
```

```
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Oleh karena itu, pemetaan nilai ke kunci berfungsi untuk anggota enum numerik, tetapi tidak untuk anggota enum string:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]);  //  Element  implicitly  has  an
'any' type because index expression is not of type 'number'.
```

Enum ambient

Enum ambient dalam TypeScript adalah jenis enum yang didefinisikan dalam berkas deklarasi (`*.d.ts`) tanpa implementasi terkait. Enum ini memungkinkan Anda menentukan sekumpulan konstanta bernama yang dapat digunakan dengan aman secara tipe di berbagai berkas tanpa harus mengimpor detail implementasi di setiap berkas.

Anggota terkomputasi dan konstan

Dalam TypeScript, anggota terkomputasi adalah anggota Enum yang nilainya dihitung saat runtime, sedangkan anggota konstan adalah anggota yang nilainya ditetapkan pada waktu kompilasi dan tidak dapat diubah saat runtime. Anggota terkomputasi diperbolehkan dalam Enum biasa, sedangkan anggota konstan diperbolehkan dalam enum biasa maupun const enum.

```
// Constant members
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6 generation at compilation time

// Computed members
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // random number generated at run time
```

Enum dinyatakan sebagai union yang terdiri dari tipe-tipe anggotanya. Nilai setiap anggota dapat ditentukan melalui ekspresi konstan atau nonkonstan, dengan anggota yang memiliki nilai konstan diberi tipe literal. Sebagai ilustrasi, perhatikan deklarasi tipe E dan subtipenya E.A, E.B, dan E.C. Dalam kasus ini, E merepresentasikan union E.A | E.B | E.C.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Numeric literal
    B = 'bar', // String literal
```

```
    C = identity(42), // Opaque computed
}

console.log(E.C); //42
```

Penyempitan

Penyempitan TypeScript adalah proses mempersempit tipe suatu variabel di dalam blok kondisional. Hal ini berguna ketika bekerja dengan tipe union, ketika sebuah variabel dapat memiliki lebih dari satu tipe.

TypeScript mengenali beberapa cara untuk mempersempit tipe:

Type guard typeof

Type guard typeof adalah salah satu type guard khusus dalam TypeScript yang memeriksa tipe suatu variabel berdasarkan tipe bawaan JavaScript-nya.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x is number
    }
    return -1;
};
```

Penyempitan berdasarkan truthiness

Penyempitan berdasarkan truthiness dalam TypeScript bekerja dengan memeriksa apakah suatu variabel bersifat truthy atau falsy untuk mempersempit tipenya sebagaimana mestinya.

```
const toUpperCase = (name: string | null) => {
    if (name) {
```

```
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

Penyempitan berdasarkan kesetaraan

Penyempitan berdasarkan kesetaraan dalam TypeScript bekerja dengan memeriksa apakah suatu variabel sama dengan nilai tertentu atau tidak, untuk mempersempit tipenya sebagaimana mestinya.

Penyempitan ini digunakan bersama statement `switch` dan operator kesetaraan seperti `===`, `!==`, `==`, dan `!=` untuk mempersempit tipe.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

Penyempitan dengan Operator In

Penyempitan Operator `in` dalam TypeScript adalah cara untuk mempersempit tipe suatu variabel berdasarkan apakah suatu properti ada di dalam tipe variabel tersebut.

```
type Dog = {
    name: string;
    breed: string;
};
```

```
type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

Penyempitan instanceof

Penyempitan dengan operator `instanceof` dalam TypeScript adalah cara untuk mempersempit tipe suatu variabel berdasarkan fungsi constructor-nya, dengan memeriksa apakah suatu objek merupakan instance dari class atau interface tertentu.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
```

```
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

Penetapan

Penyempitan TypeScript menggunakan penetapan adalah cara untuk mempersempit tipe suatu variabel berdasarkan nilai yang ditetapkan kepadanya. Ketika suatu variabel diberi nilai, TypeScript menginferensi tipenya berdasarkan nilai yang ditetapkan, lalu mempersempit tipe variabel agar sesuai dengan tipe yang diinferensi.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

Analisis Alur Kontrol

Analisis Alur Kontrol dalam TypeScript adalah cara untuk menganalisis alur kode secara statis guna menginferensi tipe variabel, yang memungkinkan compiler mempersempit tipe variabel tersebut sesuai kebutuhan berdasarkan hasil analisis.

Sebelum TypeScript 4.4, analisis alur kode hanya diterapkan pada kode di dalam statement if, tetapi mulai TypeScript 4.4, analisis ini juga dapat diterapkan pada ekspresi kondisional

dan akses properti diskriminan yang dirujuk secara tidak langsung melalui variabel const.

Sebagai contoh:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

Beberapa contoh ketika penyempitan tidak terjadi:

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
        x.length; // Error, no narrowing because isString it is
not const
    }
};

const f6 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
```

```
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
        obj.foo; // Error, no narrowing because obj is assigned
in function body
    }
};
```

Catatan: Hingga lima tingkat indireksi dianalisis dalam ekspresi kondisional.

Predikat Tipe

Predikat Tipe dalam TypeScript adalah fungsi yang mengembalikan nilai boolean dan digunakan untuk mempersempit tipe suatu variabel menjadi tipe yang lebih spesifik.

```
const isString = (value: unknown): value is string => typeof
value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

TypeScript 5.5 secara otomatis menginferensi predikat tipe (seperti `x is T`) dalam fungsi seperti `.filter`, sehingga TypeScript mengetahui ketika nilai seperti `undefined` dihapus—menghasilkan tipe yang lebih presisi dan lebih sedikit error. Ini berlaku untuk pemeriksaan yang jelas (misalnya, `x !== undefined`), tetapi tidak untuk pemeriksaan ambigu seperti `!!x`.

```ts
const nums = [1, null, 2].filter(x => x !== null);
```

Union Terdiskriminasi

Union Terdiskriminasi dalam TypeScript adalah jenis tipe union yang menggunakan properti umum, yang dikenal sebagai diskriminan, untuk mempersempit kumpulan kemungkinan tipe dalam union tersebut.

```ts
type Square = {
    kind: 'square'; // Discriminant
    size: number;
};

type Circle = {
    kind: 'circle'; // Discriminant
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

Tipe never

Ketika sebuah variabel dipersempit menjadi tipe yang tidak dapat memuat nilai apa pun, compiler TypeScript akan menginferensi bahwa variabel tersebut harus bertipe `never`. Hal ini karena tipe `never` merepresentasikan nilai yang tidak akan pernah dapat dihasilkan.

```
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
          // val has type never here because it can never be
anything other than a string or a number
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

Pemeriksaan Kelengkapan

Pemeriksaan kelengkapan adalah fitur dalam TypeScript yang memastikan semua kemungkinan kasus dari union terdiskriminasi ditangani dalam statement `switch` atau statement `if`.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
```

```
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
                console.log(exhaustiveCheck); // This line will
never be executed
    }
};
```

Tipe `never` digunakan untuk memastikan bahwa kasus default bersifat exhaustive dan bahwa TypeScript akan menghasilkan error jika nilai baru ditambahkan ke tipe Direction tanpa ditangani dalam statement switch.

Tipe Objek

Dalam TypeScript, tipe objek mendeskripsikan bentuk sebuah objek. Tipe ini menentukan nama dan tipe properti objek, serta apakah properti tersebut wajib atau opsional.

Dalam TypeScript, Anda dapat mendefinisikan tipe objek dengan dua cara utama:

Interface mendefinisikan bentuk objek dengan menentukan nama dan tipe properti, serta apakah properti tersebut opsional.

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

Type alias, serupa dengan interface, mendefinisikan bentuk sebuah objek. Namun, type alias juga dapat membuat tipe

kustom baru yang didasarkan pada tipe yang sudah ada atau kombinasi dari tipe-tipe yang sudah ada. Hal ini mencakup pendefinisian tipe union, tipe intersection, dan tipe kompleks lainnya.

```
type Point = {
    x: number;
    y: number;
};
```

Tipe juga dapat didefinisikan secara anonim:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

Tipe Tuple (Anonim)

Tipe Tuple adalah tipe yang merepresentasikan array dengan jumlah elemen tetap beserta tipe yang sesuai untuk setiap elemennya. Tipe tuple memberlakukan jumlah elemen tertentu dan masing-masing tipenya dalam urutan tetap. Tipe tuple berguna ketika Anda ingin merepresentasikan kumpulan nilai dengan tipe tertentu dan posisi setiap elemen dalam array memiliki makna tertentu.

```
type Point = [number, number];
```

Tipe Tuple Bernama (Berlabel)

Tipe tuple dapat menyertakan label atau nama opsional untuk setiap elemen. Label ini digunakan untuk keterbacaan dan dukungan tooling, serta tidak memengaruhi operasi yang dapat Anda lakukan dengannya.

```
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Named Tuple plus Anonymous Tuple
```

Tuple dengan Panjang Tetap

Tuple dengan Panjang Tetap adalah jenis tuple khusus yang memberlakukan jumlah elemen tetap dengan tipe tertentu, dan tidak mengizinkan perubahan apa pun pada panjang tuple setelah didefinisikan.

Tuple dengan Panjang Tetap berguna ketika Anda perlu merepresentasikan kumpulan nilai dengan jumlah elemen dan tipe tertentu, serta ingin memastikan bahwa panjang dan tipe tuple tidak dapat berubah secara tidak sengaja.

```
const x = [10, 'hello'] as const;
x.push(2); // Error
```

Tipe Union

Tipe Union adalah tipe yang merepresentasikan nilai yang dapat berupa salah satu dari beberapa tipe. Tipe Union dinyatakan menggunakan simbol `|` di antara setiap tipe yang mungkin.

```
let x: string | number;
x = 'hello'; // Valid
x = 123; // Valid
```

Tipe Intersection

Tipe Intersection adalah tipe yang merepresentasikan nilai yang memiliki semua properti dari dua atau lebih tipe. Tipe Intersection dinyatakan menggunakan simbol `&` di antara setiap tipe.

```
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Intersection

const j: J = {
    a: 'a',
    b: 'b',
};
```

Pengindeksan Tipe

Pengindeksan tipe mengacu pada kemampuan untuk mendefinisikan tipe yang dapat diindeks oleh kunci yang tidak diketahui sebelumnya, dengan menggunakan index signature untuk menentukan tipe bagi properti yang tidak dideklarasikan secara eksplisit.

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Returns a
```

Tipe dari Nilai

Tipe dari Nilai dalam TypeScript mengacu pada inferensi otomatis suatu tipe dari nilai atau ekspresi melalui inferensi tipe.

```
const x = 'x'; // TypeScript infers 'x' as a string literal with 'const' (immutable), but widens it to 'string' with 'let' (reassignable).
```

Tipe dari Nilai Kembalian Fungsi

Tipe dari Nilai Kembalian Fungsi mengacu pada kemampuan untuk secara otomatis menginferensi tipe kembalian suatu fungsi berdasarkan implementasinya. Hal ini memungkinkan TypeScript menentukan tipe nilai yang dikembalikan oleh fungsi tanpa anotasi tipe eksplisit.

```
const add = (x: number, y: number) => x + y; // TypeScript can infer that the return type of the function is a number
```

Tipe dari Modul

Tipe dari Modul mengacu pada kemampuan untuk menggunakan nilai yang diekspor sebuah modul guna menginferensi tipenya secara otomatis. Ketika sebuah modul mengekspor nilai dengan tipe tertentu, TypeScript dapat menggunakan informasi tersebut untuk secara otomatis menginferensi tipe nilai itu ketika diimpor ke modul lain.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r is number
```

Mapped Type

Mapped Type dalam TypeScript memungkinkan Anda membuat tipe baru berdasarkan tipe yang sudah ada dengan mentransformasi setiap properti menggunakan fungsi pemetaan. Dengan memetakan tipe yang sudah ada, Anda dapat membuat tipe baru yang merepresentasikan informasi yang sama dalam format berbeda. Untuk membuat mapped type, Anda mengakses properti tipe yang sudah ada menggunakan operator `keyof`, lalu mengubahnya untuk menghasilkan tipe baru. Dalam contoh berikut:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

Kita mendefinisikan `MyMappedType` untuk memetakan properti-properti `T`, sehingga menghasilkan tipe baru dengan setiap properti berupa array dari tipe aslinya. Dengan menggunakan ini, kita membuat `MyNewType` untuk merepresentasikan informasi yang sama seperti `MyType`, tetapi setiap propertinya berupa array.

Modifier Mapped Type

Modifier Mapped Type dalam TypeScript memungkinkan transformasi properti di dalam tipe yang sudah ada:

- `readonly` atau `+readonly`: Ini menjadikan properti dalam mapped type sebagai read-only.
- `-readonly`: Ini memungkinkan properti dalam mapped type bersifat mutable.
- `?`: Ini menetapkan properti dalam mapped type sebagai opsional.

Contoh:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // All properties marked as read-only

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // All properties marked as mutable

type MyPartial<T> = { [P in keyof T]?: T[P] }; // All properties marked as optional
```

Tipe Kondisional

Tipe Kondisional adalah cara untuk membuat tipe yang bergantung pada suatu kondisi, ketika tipe yang akan dibuat ditentukan berdasarkan hasil kondisi tersebut. Tipe ini didefinisikan menggunakan kata kunci `extends` dan operator ternary untuk memilih salah satu dari dua tipe secara kondisional.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;
```

```
type IsMyArrayAnArray = IsArray<typeof myArray>; // Type true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Type false
```

Tipe Kondisional Distributif

Tipe Kondisional Distributif adalah fitur yang memungkinkan suatu tipe didistribusikan pada sebuah union tipe dengan menerapkan transformasi pada setiap member union secara individual. Hal ini dapat sangat berguna ketika bekerja dengan mapped type atau higher-order type.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number | boolean | null
```

Inferensi Tipe infer dalam Tipe Kondisional

Kata kunci `infer` digunakan dalam tipe kondisional untuk menginferensi (mengekstrak) tipe parameter generik dari tipe yang bergantung padanya. Hal ini memungkinkan Anda menulis definisi tipe yang lebih fleksibel dan dapat digunakan kembali.

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

Tipe Kondisional yang Telah Didefinisikan

Dalam TypeScript, tipe kondisional yang telah didefinisikan adalah tipe kondisional bawaan yang disediakan oleh bahasa

tersebut. Tipe-tipe ini dirancang untuk melakukan transformasi tipe yang umum berdasarkan karakteristik tipe tertentu.

`Exclude<UnionType, ExcludedType>`: Tipe ini menghapus semua tipe dari Type yang dapat ditetapkan ke ExcludedType.

`Extract<Type, Union>`: Tipe ini mengekstrak semua tipe dari Union yang dapat ditetapkan ke Type.

`NonNullable<Type>`: Tipe ini menghapus null dan undefined dari Type.

`ReturnType<Type>`: Tipe ini mengekstrak tipe kembalian dari Type yang berupa fungsi.

`Parameters<Type>`: Tipe ini mengekstrak tipe parameter dari Type yang berupa fungsi.

`Required<Type>`: Tipe ini membuat semua properti dalam Type menjadi wajib.

`Partial<Type>`: Tipe ini membuat semua properti dalam Type menjadi opsional.

`Readonly<Type>`: Tipe ini membuat semua properti dalam Type menjadi hanya-baca.

Tipe Union Template

Tipe union templat dapat digunakan untuk menggabungkan dan memanipulasi teks di dalam sistem tipe, misalnya:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
```

```ts
type ProductId = `id-${Products}-${Status}`; // "id-p1-active" | "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Tipe Any

Tipe `any` adalah tipe khusus (supertipe universal) yang dapat digunakan untuk merepresentasikan nilai dari tipe apa pun (primitif, objek, array, fungsi, error, simbol). Tipe ini sering digunakan dalam situasi ketika tipe suatu nilai tidak diketahui pada waktu kompilasi, atau saat bekerja dengan nilai dari API atau pustaka eksternal yang tidak memiliki definisi tipe TypeScript.

Dengan menggunakan tipe `any`, Anda memberi tahu compiler TypeScript bahwa nilai harus direpresentasikan tanpa batasan apa pun. Untuk memaksimalkan keamanan tipe dalam kode Anda, pertimbangkan hal-hal berikut:

- Batasi penggunaan `any` pada kasus tertentu ketika tipenya benar-benar tidak diketahui.
- Jangan mengembalikan tipe `any` dari suatu fungsi karena hal ini melemahkan keamanan tipe dalam kode yang menggunakannya.
- Alih-alih menggunakan `any`, gunakan `@ts-ignore` jika Anda perlu membungkam compiler.

```ts
let value: any;
value = true; // Valid
value = 7; // Valid
```

Tipe Unknown

Dalam TypeScript, tipe unknown merepresentasikan nilai yang tipenya tidak diketahui. Tidak seperti tipe any, yang memungkinkan nilai dari tipe apa pun, unknown memerlukan pemeriksaan atau asersi tipe sebelum dapat digunakan dengan cara tertentu. Oleh karena itu, operasi apa pun tidak diizinkan pada unknown tanpa terlebih dahulu melakukan asersi atau narrowing ke tipe yang lebih spesifik.

Tipe unknown hanya dapat ditetapkan ke any dan unknown itu sendiri, serta merupakan alternatif yang aman secara tipe untuk any.

```
let value: unknown;

let value1: unknown = value; // Valid
let value2: any = value; // Valid
let value3: boolean = value; // Invalid
let value4: number = value; // Invalid

const add = (a: unknown, b: unknown): number | undefined =>
     typeof a === 'number' && typeof b === 'number' ? a + b :
undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Tipe Void

Tipe void digunakan untuk menunjukkan bahwa suatu fungsi tidak mengembalikan nilai.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

Tipe Never

Tipe `never` merepresentasikan nilai yang tidak pernah terjadi. Tipe ini digunakan untuk menunjukkan fungsi atau ekspresi yang tidak pernah mengembalikan nilai atau melempar error.

Misalnya, sebuah perulangan tak terbatas:

```
const infiniteLoop = (): never => {
    while (true) {
        // do something
    }
};
```

Melempar error:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

Tipe `never` berguna untuk memastikan keamanan tipe dan mendeteksi potensi error dalam kode Anda. Tipe ini membantu TypeScript menganalisis dan menyimpulkan tipe yang lebih presisi ketika digunakan bersama tipe lain dan pernyataan alur kontrol, misalnya:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // move up
            break;
        case 'down':
            // move down
            break;
        default:
            const exhaustiveCheck: never = direction;
                        throw new Error(`Unhandled direction:
${exhaustiveCheck}`);
```

```
    }
};
```

Interface dan Type

Sintaks Umum

Dalam TypeScript, interface mendefinisikan struktur objek dengan menentukan nama dan tipe properti atau metode yang harus dimiliki sebuah objek. Sintaks umum untuk mendefinisikan interface dalam TypeScript adalah sebagai berikut:

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

Demikian pula untuk definisi tipe:

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` atau `type TypeName`: Mendefinisikan nama interface. `property1: Type1`: Menentukan properti interface beserta tipe yang sesuai. Beberapa properti dapat didefinisikan, masing-masing dipisahkan oleh titik koma. `method1(arg1: ArgType1, arg2: ArgType2): ReturnType;`: Menentukan metode interface. Metode didefinisikan dengan namanya, diikuti daftar

parameter dalam tanda kurung dan tipe kembalian. Beberapa metode dapat didefinisikan, masing-masing dipisahkan oleh titik koma.

Contoh interface:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

Contoh type:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

Dalam TypeScript, tipe digunakan untuk mendefinisikan bentuk data dan menerapkan pemeriksaan tipe. Ada beberapa sintaks umum untuk mendefinisikan tipe dalam TypeScript, bergantung pada kasus penggunaan tertentu. Berikut beberapa contohnya:

Tipe Dasar

```
let myNumber: number = 123; // number type
let myBoolean: boolean = true; // boolean type
let myArray: string[] = ['a', 'b']; // array of strings
let myTuple: [string, number] = ['a', 123]; // tuple
```

Objek dan Interface

```
const x: { name: string; age: number } = { name: 'Simon', age:
7 };
```

Tipe Union dan Intersection

```
type MyType = string | number; // Union type
let myUnion: MyType = 'hello'; // Can be a string
myUnion = 123; // Or a number

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Intersection type
let myCombined: CombinedType = { name: 'John', age: 25 }; //
Object with both name and age properties
```

Tipe Primitif Bawaan

TypeScript memiliki beberapa tipe primitif bawaan yang dapat digunakan untuk mendefinisikan variabel, parameter fungsi, dan tipe kembalian:

- `number`: Merepresentasikan nilai numerik, termasuk bilangan bulat dan bilangan floating-point.
- `string`: Merepresentasikan data tekstual
- `boolean`: Merepresentasikan nilai logika, yang dapat berupa true atau false.
- `null`: Merepresentasikan ketiadaan nilai.
- `undefined`: Merepresentasikan nilai yang belum ditetapkan atau belum didefinisikan.
- `symbol`: Merepresentasikan pengidentifikasi unik. Symbol biasanya digunakan sebagai kunci untuk properti objek.
- `bigint`: Merepresentasikan bilangan bulat dengan presisi arbitrer.
- `any`: Merepresentasikan tipe dinamis atau tidak diketahui. Variabel bertipe any dapat menampung nilai dari tipe apa pun dan melewati pemeriksaan tipe.

- `void`: Merepresentasikan ketiadaan tipe apa pun. Tipe ini umumnya digunakan sebagai tipe kembalian fungsi yang tidak mengembalikan nilai.
- `never`: Merepresentasikan tipe untuk nilai yang tidak pernah terjadi. Tipe ini biasanya digunakan sebagai tipe kembalian fungsi yang melempar error atau memasuki perulangan tak terbatas.

Objek JS Bawaan yang Umum

TypeScript adalah superset dari JavaScript; TypeScript mencakup semua objek JavaScript bawaan yang umum digunakan. Anda dapat menemukan daftar lengkap objek ini di situs dokumentasi Mozilla Developer Network (MDN): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Berikut adalah daftar beberapa objek JavaScript bawaan yang umum digunakan:

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map

- Set
- Promise
- Intl

Overload

Overload fungsi dalam TypeScript memungkinkan Anda mendefinisikan beberapa signature fungsi untuk satu nama fungsi, sehingga Anda dapat mendefinisikan fungsi yang bisa dipanggil dengan berbagai cara. Berikut contohnya:

```
// Overloads
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Implementation
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Valid
sayHi(['aa', 'bb']); // Valid
```

Berikut contoh lain penggunaan overload fungsi di dalam sebuah `class`:

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
```

```
    }

    // overload
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // implementation
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
                    return  name.map(name  =>  `${this.message},
${name}!`);
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

Penggabungan dan Ekstensi

Penggabungan dan ekstensi mengacu pada dua konsep berbeda yang berkaitan dengan penggunaan tipe dan interface.

Penggabungan memungkinkan Anda menggabungkan beberapa deklarasi dengan nama yang sama menjadi satu definisi, misalnya ketika Anda mendefinisikan interface dengan nama yang sama beberapa kali:

```
interface X {
    a: string;
}

interface X {
    b: number;
}
```

```
const person: X = {
    a: 'a',
    b: 7,
};
```

Ekstensi mengacu pada kemampuan untuk memperluas atau mewarisi tipe atau interface yang sudah ada guna membuat tipe atau interface baru. Ini adalah mekanisme untuk menambahkan properti atau metode tambahan ke tipe yang sudah ada tanpa mengubah definisi aslinya. Contoh:

```
interface Animal {
    name: string;
    eat(): void;
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

Perbedaan antara Type dan Interface

Penggabungan deklarasi (augmentasi):

Interface mendukung penggabungan deklarasi, yang berarti Anda dapat mendefinisikan beberapa interface dengan nama

yang sama dan TypeScript akan menggabungkannya menjadi satu interface dengan gabungan properti dan metode. Di sisi lain, type tidak mendukung penggabungan deklarasi. Hal ini dapat membantu ketika Anda ingin menambahkan fungsionalitas tambahan atau menyesuaikan tipe yang sudah ada tanpa mengubah definisi asli maupun menambal tipe yang hilang atau keliru.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

Memperluas tipe/interface lain:

Baik type maupun interface dapat memperluas tipe/interface lain, tetapi sintaksnya berbeda. Pada interface, Anda menggunakan kata kunci `extends` untuk mewarisi properti dan metode dari interface lain. Namun, sebuah interface tidak dapat memperluas tipe kompleks seperti tipe union.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
```

```
    y: 123,
    z: 'z',
};
```

Untuk type, Anda menggunakan operator `&` untuk menggabungkan beberapa tipe menjadi satu tipe (intersection).

```
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

Tipe Union dan Intersection:

Type lebih fleksibel dalam mendefinisikan Tipe Union dan Intersection. Dengan kata kunci `type`, Anda dapat dengan mudah membuat tipe union menggunakan operator `|` dan tipe intersection menggunakan operator `&`. Meskipun interface juga dapat merepresentasikan tipe union secara tidak langsung, interface tidak memiliki dukungan bawaan untuk tipe intersection.

```
type Department = 'dep-x' | 'dep-y'; // Union

type Person = {
    name: string;
    age: number;
```

```
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Intersection
```

Contoh dengan interface:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // Union of interfaces
```

Class

Sintaks Umum Class

Kata kunci `class` digunakan dalam TypeScript untuk mendefinisikan sebuah class. Berikut adalah contohnya:

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
        console.log(
```

```
                `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
```

Kata kunci `class` digunakan untuk mendefinisikan class bernama `Person`.

Class tersebut memiliki dua properti private: `name` bertipe `string` dan `age` bertipe `number`.

Constructor didefinisikan menggunakan kata kunci `constructor`. Constructor menerima `name` dan `age` sebagai parameter lalu menetapkannya ke properti yang sesuai.

Class tersebut memiliki metode `public` bernama `sayHi` yang mencatat pesan sapaan.

Untuk membuat instance dari sebuah class dalam TypeScript, Anda dapat menggunakan kata kunci `new` diikuti nama class, lalu tanda kurung `()`. Misalnya:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Output: Hello, my name is John Doe and I
am 25 years old.
```

Constructor

Constructor adalah metode khusus dalam sebuah class yang digunakan untuk menginisialisasi properti objek saat instance class dibuat.

```
class Person {
    public name: string;
    public age: number;
```

```
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
            `Hello, my name is ${this.name} and I'm ${this.age}
years old.`
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

Constructor dapat di-overload menggunakan sintaks berikut:

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

Dalam TypeScript, beberapa overload constructor dapat didefinisikan, tetapi hanya boleh ada satu implementasi yang harus kompatibel dengan semua overload; hal ini dapat dilakukan dengan menggunakan parameter opsional.

```
class Person {
    name: string;
    age: number;

    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

Constructor Private dan Protected

Dalam TypeScript, constructor dapat ditandai sebagai `private` atau `protected`, yang membatasi aksesibilitas dan penggunaannya.

Constructor private: Hanya dapat dipanggil dari dalam class itu sendiri. Constructor private sering digunakan dalam skenario ketika Anda ingin menerapkan pola singleton atau membatasi pembuatan instance melalui metode factory di dalam class.

Constructor protected: Constructor protected berguna ketika Anda ingin membuat base class yang tidak boleh dibuat instance-nya secara langsung, tetapi dapat diperluas oleh subclass.

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// Attempting to instantiate the base class directly will result in an error
// const baseObj = new BaseClass(); // Error: Constructor of class 'BaseClass' is protected.

// Create an instance of the derived class
const derivedObj = new DerivedClass(10);
```

Modifier Akses

Modifier akses `private`, `protected`, dan `public` digunakan untuk mengontrol visibilitas dan aksesibilitas anggota class, seperti properti dan metode, dalam class TypeScript. Modifier ini

penting untuk menerapkan enkapsulasi dan menetapkan batasan dalam mengakses serta mengubah state internal sebuah class.

Modifier `private` membatasi akses ke anggota class hanya dari dalam class yang memuatnya.

Modifier `protected` mengizinkan akses ke anggota class dari dalam class yang memuatnya dan class turunannya.

Modifier `public` memberikan akses tanpa batas ke anggota class, sehingga dapat diakses dari mana saja.

Get dan Set

Getter dan setter adalah metode khusus yang memungkinkan Anda mendefinisikan perilaku akses dan perubahan khusus untuk properti class. Keduanya memungkinkan Anda mengenkapsulasi state internal sebuah objek dan menyediakan logika tambahan ketika mengambil atau menetapkan nilai properti. Dalam TypeScript, getter dan setter masing-masing didefinisikan menggunakan kata kunci `get` dan `set`. Berikut contohnya:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
```

```
    }
}
```

Auto-Accessor dalam Class

TypeScript versi 4.9 menambahkan dukungan untuk auto-accessor, sebuah fitur ECMAScript yang akan datang. Fitur ini menyerupai properti class, tetapi dideklarasikan dengan kata kunci `accessor`.

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

Auto-accessor diubah ("de-sugared") menjadi accessor `get` dan `set` private yang beroperasi pada properti yang tidak dapat diakses.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

this

Dalam TypeScript, kata kunci `this` mengacu pada instance class saat ini di dalam metode atau constructor-nya. Kata kunci ini memungkinkan Anda mengakses dan mengubah properti serta metode class dari dalam cakupannya sendiri. Kata kunci ini menyediakan cara untuk mengakses dan memanipulasi state internal sebuah objek di dalam metode objek itu sendiri.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

Properti Parameter

Properti parameter memungkinkan Anda mendeklarasikan dan menginisialisasi properti class secara langsung di dalam parameter constructor, sehingga menghindari kode boilerplate. Contoh:

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
            // The "private" and "public" keywords in the
constructor
```

```
                // automatically declare and initialize the
corresponding class properties.
    }
    public introduce(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

Class Abstrak

Class abstrak digunakan dalam TypeScript terutama untuk pewarisan. Class ini menyediakan cara untuk mendefinisikan properti dan metode umum yang dapat diwarisi oleh subclass. Hal ini berguna ketika Anda ingin mendefinisikan perilaku umum dan memastikan bahwa subclass mengimplementasikan metode tertentu. Class abstrak menyediakan cara untuk membuat hierarki class, dengan base class abstrak menyediakan interface bersama dan fungsionalitas umum untuk subclass.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
```

```
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Output: Whiskers meows.
```

Dengan Generic

Class dengan generic memungkinkan Anda mendefinisikan class yang dapat digunakan kembali dan bekerja dengan berbagai tipe.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

Decorator

Decorator menyediakan mekanisme untuk menambahkan metadata, mengubah perilaku, memvalidasi, atau memperluas fungsionalitas elemen target. Decorator adalah fungsi yang dijalankan saat runtime. Beberapa decorator dapat diterapkan pada sebuah deklarasi.

Decorator merupakan fitur eksperimental, dan contoh berikut hanya kompatibel dengan TypeScript versi 5 atau lebih baru yang menggunakan ES6.

Untuk TypeScript versi sebelum 5, decorator harus diaktifkan menggunakan properti `experimentalDecorators` dalam `tsconfig.json` Anda atau menggunakan `--experimentalDecorators` pada baris perintah (tetapi contoh berikut tidak akan berfungsi).

Beberapa kasus penggunaan umum untuk decorator meliputi:

- Mengamati perubahan properti.
- Mengamati pemanggilan metode.
- Menambahkan properti atau metode tambahan.
- Validasi runtime.
- Serialisasi dan deserialisasi otomatis.
- Logging.
- Otorisasi dan autentikasi.
- Perlindungan terhadap error.

Catatan: Decorator untuk versi 5 tidak mengizinkan dekorasi parameter.

Jenis decorator:

Decorator Class

Decorator Class berguna untuk memperluas class yang sudah ada, misalnya dengan menambahkan properti atau metode, maupun mengumpulkan instance sebuah class. Dalam contoh berikut, kita menambahkan metode `toString` yang mengubah class menjadi representasi string.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}

@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* Logs:
{"name":"Simon"}
```

```
{"kind":"class","name":"Person"}
*/
```

Decorator Properti

Decorator properti berguna untuk mengubah perilaku suatu properti, seperti mengubah nilai inisialisasi. Dalam kode berikut, terdapat skrip yang memastikan nilai sebuah properti selalu menggunakan huruf besar:

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
    @upperCase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // Logs: HELLO!
```

Decorator Metode

Decorator metode memungkinkan Anda mengubah atau meningkatkan perilaku metode. Berikut adalah contoh logger sederhana:

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
```

```
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

       function replacementMethod(this: This, ...args: Args):
Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}

new MyClass().sayHello();
```

Output log:

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

Decorator Getter dan Setter

Decorator getter dan setter memungkinkan Anda mengubah atau meningkatkan perilaku accessor class. Decorator ini berguna, misalnya, untuk memvalidasi penetapan properti. Berikut contoh sederhana decorator getter:

```
function range<This, Return extends number>(min: number, max:
number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
}

class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Valid: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // Throw: Invalid!
```

Metadata Decorator

Metadata Decorator menyederhanakan proses penerapan dan penggunaan metadata oleh decorator dalam class apa pun. Decorator dapat mengakses properti metadata baru pada objek context, yang dapat berfungsi sebagai key untuk primitif maupun objek. Informasi metadata dapat diakses pada class melalui `Symbol.metadata`.

Metadata dapat digunakan untuk berbagai tujuan, seperti debugging, serialisasi, atau dependency injection dengan decorator.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Simple polyfill

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
    | ClassMethodDecoratorContext; // Context contains property metadata: DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // Set the metadata object with a primitive value
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
```

```
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Get metadata
information

console.log(JSON.stringify(metadata));                    //
{"bar":true,"baz":true,"foo":true}
```

Pewarisan

Pewarisan mengacu pada mekanisme yang memungkinkan sebuah class mewarisi properti dan metode dari class lain, yang dikenal sebagai base class atau superclass. Derived class, yang juga disebut child class atau subclass, dapat memperluas dan mengkhususkan fungsionalitas base class dengan menambahkan properti dan metode baru atau meng-override yang sudah ada.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
```

```
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Create an instance of the base class
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Create an instance of the derived class
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript tidak mendukung pewarisan berganda dalam pengertian tradisional, tetapi memungkinkan pewarisan dari satu base class. TypeScript mendukung beberapa interface. Sebuah interface dapat mendefinisikan kontrak untuk struktur objek, dan sebuah class dapat mengimplementasikan beberapa interface. Hal ini memungkinkan sebuah class mewarisi perilaku dan struktur dari beberapa sumber.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
```

```
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

Kata kunci `class` dalam TypeScript, sama seperti dalam JavaScript, sering disebut sebagai syntactic sugar. Kata kunci ini diperkenalkan dalam ECMAScript 2015 (ES6) untuk menawarkan sintaks yang lebih familier dalam membuat dan bekerja dengan objek menggunakan pendekatan berbasis class. Namun, penting untuk dicatat bahwa TypeScript, sebagai superset dari JavaScript, pada akhirnya dikompilasi menjadi JavaScript, yang secara mendasar tetap berbasis prototype.

Anggota Statis

TypeScript memiliki anggota statis. Untuk mengakses anggota statis sebuah class, Anda dapat menggunakan nama class yang diikuti titik tanpa perlu membuat objek.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

Inisialisasi properti

Ada beberapa cara untuk menginisialisasi properti sebuah class dalam TypeScript:

Inline:

Dalam contoh berikut, nilai awal ini akan digunakan saat instance class dibuat.

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

Di dalam constructor:

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

Menggunakan parameter constructor:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
            // There is no need to assign the values to the
properties explicitly.
    }
    log() {
```

```
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

Overload metode

Overload metode memungkinkan sebuah class memiliki beberapa metode dengan nama yang sama, tetapi dengan tipe parameter atau jumlah parameter yang berbeda. Hal ini memungkinkan kita memanggil sebuah metode dengan berbagai cara berdasarkan argumen yang diteruskan.

```
class MyClass {
    add(a: number, b: number): number; // Overload signature 1
    add(a: string, b: string): string; // Overload signature 2

      add(a: number | string, b: number | string): number |
string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}

const r = new MyClass();
console.log(r.add(10, 5)); // Logs 15
```

Generic

Generik memungkinkan Anda membuat komponen dan fungsi yang dapat digunakan kembali dan bekerja dengan berbagai tipe. Dengan generik, Anda dapat memparametrisasi tipe, fungsi, dan antarmuka, sehingga semuanya dapat beroperasi pada tipe yang berbeda tanpa perlu menentukannya secara eksplisit terlebih dahulu.

Generik memungkinkan Anda membuat kode yang lebih fleksibel dan dapat digunakan kembali.

Tipe Generic

Untuk mendefinisikan tipe generik, Anda menggunakan tanda kurung sudut (<>) guna menentukan parameter tipe, misalnya:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

Class Generic

Generik juga dapat diterapkan pada kelas. Dengan cara ini, kelas dapat bekerja dengan berbagai tipe menggunakan parameter tipe. Hal ini berguna untuk membuat definisi kelas yang dapat digunakan kembali dan beroperasi pada tipe data yang berbeda sambil tetap mempertahankan keamanan tipe.

```
class Container<T> {
    private item: T;
```

```
    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

Batasan Generic

Parameter generik dapat dibatasi menggunakan kata kunci `extends` yang diikuti oleh tipe atau antarmuka yang harus dipenuhi oleh parameter tipe tersebut.

Dalam contoh berikut, `T` harus memiliki properti `length` dengan tipe yang tepat agar valid:

```
const printLen = <T extends { length: number }>(value: T): void
=> {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // Invalid
```

Salah satu fitur generik penting yang diperkenalkan pada versi 3.4 RC adalah inferensi tipe fungsi tingkat tinggi, yang meneruskan argumen tipe generik:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Fungsionalitas ini mempermudah pemrograman bergaya point-free yang aman secara tipe dan umum digunakan dalam pemrograman fungsional.

Penyempitan kontekstual generic

Penyempitan kontekstual untuk generik adalah mekanisme dalam TypeScript yang memungkinkan kompiler mempersempit tipe parameter generik berdasarkan konteks penggunaannya. Mekanisme ini berguna saat bekerja dengan tipe generik dalam pernyataan kondisional:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Value is narrowed down to type 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // Value is narrowed down to type 'number'
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

Tipe Struktural yang Dihapus

Dalam TypeScript, objek tidak harus cocok dengan tipe tertentu secara persis. Misalnya, jika kita membuat objek yang memenuhi persyaratan suatu antarmuka, kita dapat menggunakan objek tersebut di tempat yang memerlukan antarmuka itu, meskipun tidak ada hubungan eksplisit di antara keduanya. Contoh:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
};

log(obj); // Valid
```

Namespace

Dalam TypeScript, namespace digunakan untuk mengatur kode ke dalam wadah logis, mencegah benturan penamaan, dan menyediakan cara untuk mengelompokkan kode yang saling berkaitan. Penggunaan kata kunci `export` memungkinkan akses ke namespace dari luar modul.

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
```

```
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

Symbol

Simbol adalah tipe data primitif yang merepresentasikan nilai yang tidak dapat diubah serta dijamin unik secara global selama masa hidup program.

Simbol dapat digunakan sebagai kunci untuk properti objek dan menyediakan cara untuk membuat properti yang tidak dapat dienumerasi.

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

Dalam WeakMap dan WeakSet, simbol kini dapat digunakan sebagai kunci.

Direktif Triple-Slash

Direktif triple-slash adalah komentar khusus yang memberikan instruksi kepada kompiler tentang cara memproses sebuah berkas. Direktif ini diawali dengan tiga garis miring berturut-turut (`///`), biasanya ditempatkan di bagian atas berkas TypeScript, dan tidak berpengaruh terhadap perilaku saat runtime.

Direktif triple-slash digunakan untuk mereferensikan dependensi eksternal, menentukan perilaku pemuatan modul, mengaktifkan atau menonaktifkan fitur kompiler tertentu, dan lainnya. Beberapa contohnya:

Mereferensikan berkas deklarasi:

```
/// <reference path="path/to/declaration/file.d.ts" />
```

Menunjukkan format modul:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Mengaktifkan opsi kompiler, dalam contoh berikut mode ketat:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

Manipulasi Tipe

Membuat Tipe dari Tipe

Tipe baru dapat dibuat dengan menyusun, memanipulasi, atau mentransformasi tipe yang sudah ada.

Tipe Interseksi (`&`):

Memungkinkan Anda menggabungkan beberapa tipe menjadi satu tipe:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Intersection of A and B
const obj: C = { foo: 42, bar: 'hello' };
```

Tipe Union (|):

Memungkinkan Anda mendefinisikan sebuah tipe yang dapat berupa salah satu dari beberapa tipe:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Mapped Type:

Memungkinkan Anda mentransformasi properti dari tipe yang sudah ada untuk membuat tipe baru:

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Properties become read-only
```

Tipe kondisional:

Memungkinkan Anda membuat tipe berdasarkan beberapa kondisi:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P : never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

Tipe Akses Terindeks

Dalam TypeScript, tipe properti di dalam tipe lain dapat diakses dan dimanipulasi menggunakan indeks, `Type[Key]`.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

Tipe Utilitas

Beberapa tipe utilitas bawaan dapat digunakan untuk memanipulasi tipe. Berikut adalah daftar tipe yang paling umum digunakan:

Awaited<T>

Membentuk tipe yang secara rekursif membuka pembungkus tipe `Promise`.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

Membentuk tipe dengan semua properti T ditetapkan sebagai opsional.

```
type Person = {
    name: string;
```

```
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?:
number | undefined; }
```

Required<T>

Membentuk tipe dengan semua properti T ditetapkan sebagai wajib.

```
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

Membentuk tipe dengan semua properti T ditetapkan sebagai hanya-baca.

```
type Person = {
    name: string;
    age: number;
};

type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Invalid
```

Record<K, T>

Membentuk tipe dengan sekumpulan properti K bertipe T.

```ts
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

Membentuk tipe dengan memilih properti K yang ditentukan dari T.

```ts
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

Membentuk tipe dengan menghilangkan properti K yang ditentukan dari T.

```ts
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

Membentuk tipe dengan mengecualikan semua nilai bertipe U dari T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

Membentuk tipe dengan mengekstrak semua nilai bertipe U dari T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

Membentuk tipe dengan mengecualikan null dan undefined dari T.

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

Mengekstrak tipe parameter dari tipe fungsi T.

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

Mengekstrak tipe parameter dari tipe fungsi konstruktor T.

```
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof
Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

Mengekstrak tipe kembalian dari tipe fungsi T.

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

Mengekstrak tipe instans dari tipe kelas T.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}

type PersonInstance = InstanceType<typeof Person>;
```

```
const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

Mengekstrak tipe parameter `this` dari tipe fungsi T.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type PersonThisType = ThisParameterType<Person['greet']>; // Person
```

OmitThisParameter<T>

Menghapus parameter `this` dari tipe fungsi T.

```
function capitalize(this: String) {
                        return      this[0].toUpperCase()      + this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; // () => string
```

ThisType<T>

Berfungsi sebagai penanda untuk tipe `this` kontekstual.

```
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } & ThisType<Logger> = {
```

```
    hello: function () {
        this.log('some error'); // Valid as "log" is a part of
"this".
        this.update(); // Invalid
    },
};
```

Uppercase<T>

Mengubah nama tipe masukan T menjadi huruf besar.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

Mengubah nama tipe masukan T menjadi huruf kecil.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

Mengubah huruf pertama nama tipe masukan T menjadi huruf besar.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

Mengubah huruf pertama nama tipe masukan T menjadi huruf kecil.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer adalah tipe utilitas yang dirancang untuk memblokir inferensi tipe otomatis dalam cakupan fungsi generik.

Contoh:

```
// Automatic inference of types within the scope of a generic function.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Type here is ("a" | "b" | "c")[]
```

Dengan NoInfer:

```
// Example function that uses NoInfer to prevent type inference
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Error: Type Argument of type '"c"' is not assignable to parameter of type '"a" | "b"'.
```

Lain-lain

Error dan Penanganan Exception

TypeScript memungkinkan Anda menangkap dan menangani galat menggunakan mekanisme penanganan galat standar JavaScript:

Blok Try-Catch-Finally:

```
try {
    // Code that might throw an error
} catch (error) {
    // Handle the error
```

```
} finally {
    // Code that always executes, finally is optional
}
```

Anda juga dapat menangani berbagai tipe galat:

```
try {
    // Code that might throw different types of errors
} catch (error) {
    if (error instanceof TypeError) {
        // Handle TypeError
    } else if (error instanceof RangeError) {
        // Handle RangeError
    } else {
        // Handle other errors
    }
}
```

Tipe Galat Kustom:

Galat yang lebih spesifik dapat ditentukan dengan memperluas class `Error`:

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

Class Mixin

Class mixin memungkinkan Anda menggabungkan dan menyusun perilaku dari beberapa class ke dalam satu class. Class ini menyediakan cara untuk menggunakan kembali dan

memperluas fungsionalitas tanpa memerlukan rantai pewarisan yang mendalam.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Extend MyClass to include the behavior of Identifiable and
Selectable
interface MyClass extends Identifiable, Selectable {}

// Function to apply mixins to a class
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {

Object.getOwnPropertyNames(baseCtor.prototype).forEach(name   =>
{
            let descriptor = Object.getOwnPropertyDescriptor(
                baseCtor.prototype,
                name
            );
            if (descriptor) {
```

```
                Object.defineProperty(source.prototype, name,
descriptor);
            }
        });
    });
}

// Apply the mixins to MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

Fitur Bahasa Asinkron

Karena TypeScript merupakan superset dari JavaScript, TypeScript memiliki fitur bahasa asinkron bawaan JavaScript seperti:

Promise:

Promise adalah cara untuk menangani operasi asinkron beserta hasilnya menggunakan metode seperti `.then()` dan `.catch()` guna menangani kondisi berhasil maupun galat.

Pelajari lebih lanjut: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

Kata kunci `async`/`await` menyediakan sintaks yang tampak lebih sinkron untuk bekerja dengan Promise. Kata kunci `async` digunakan untuk mendefinisikan fungsi asinkron, sedangkan kata kunci `await` digunakan di dalam fungsi `async` untuk

menjeda eksekusi hingga sebuah Promise diselesaikan atau ditolak.

Pelajari lebih lanjut: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

API berikut didukung dengan baik di TypeScript:

Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Worker: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Worker: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Iterator dan Generator

Iterator dan generator sama-sama didukung dengan baik di TypeScript.

Iterator adalah objek yang mengimplementasikan protokol iterator, yang menyediakan cara untuk mengakses elemen dari koleksi atau urutan satu per satu. Iterator merupakan struktur yang berisi penunjuk ke elemen berikutnya dalam iterasi. Iterator memiliki metode `next()` yang mengembalikan nilai berikutnya dalam urutan beserta nilai boolean yang menunjukkan apakah urutan tersebut sudah `done`.

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

Generator adalah fungsi khusus yang didefinisikan menggunakan sintaks `function*` yang menyederhanakan pembuatan iterator. Generator menggunakan kata kunci `yield` untuk mendefinisikan urutan nilai serta secara otomatis menjeda dan melanjutkan eksekusi ketika nilai diminta.

Generator mempermudah pembuatan iterator dan sangat berguna untuk bekerja dengan urutan besar atau tak terbatas.

Contoh:

```
function* numberGenerator(start: number, end: number): Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript juga mendukung iterator asinkron dan generator asinkron.

Pelajari lebih lanjut:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

Referensi TsDocs JSDoc

Saat bekerja dengan basis kode JavaScript, Anda dapat membantu TypeScript menginferensi tipe yang tepat menggunakan komentar JSDoc beserta anotasi tambahan yang menyediakan informasi tipe.

Contoh:

```
/**
 * Computes the power of a given number
 * @constructor
 * @param {number} base - The base value of the expression
  * @param {number} exponent - The exponent value of the
expression
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10,  2);  // function power(base:  number,  exponent:
number): number
```

Dokumentasi lengkap tersedia di tautan berikut: https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

Sejak versi 3.7, definisi tipe `.d.ts` dapat dibuat dari sintaks JSDoc JavaScript. Informasi lebih lanjut dapat ditemukan di sini: https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

Paket dalam organisasi `@types` adalah konvensi penamaan paket khusus yang digunakan untuk menyediakan definisi tipe bagi pustaka atau modul JavaScript yang sudah ada. Misalnya, menggunakan:

```
npm install --save-dev @types/lodash
```

akan menginstal definisi tipe `lodash` di proyek Anda saat ini.

Untuk berkontribusi pada definisi tipe suatu paket `@types`, silakan kirim pull request ke https://github.com/DefinitelyTyped/DefinitelyTyped.

JSX

JSX (JavaScript XML) adalah ekstensi sintaks bahasa JavaScript yang memungkinkan Anda menulis kode menyerupai HTML di dalam berkas JavaScript atau TypeScript. JSX umumnya digunakan di React untuk mendefinisikan struktur HTML.

TypeScript memperluas kemampuan JSX dengan menyediakan pemeriksaan tipe dan analisis statis.

Untuk menggunakan JSX, Anda perlu menetapkan opsi kompiler `jsx` dalam berkas `tsconfig.json`. Dua opsi konfigurasi yang umum:

- `"preserve"`: menghasilkan berkas `.jsx` dengan JSX yang tidak diubah. Opsi ini memberi tahu TypeScript untuk mempertahankan sintaks JSX apa adanya dan tidak mentransformasinya selama proses kompilasi. Anda dapat menggunakan opsi ini jika memiliki alat terpisah, seperti Babel, yang menangani transformasi tersebut.
- `"react"`: mengaktifkan transformasi JSX bawaan TypeScript. `React.createElement` akan digunakan.

Semua opsi tersedia di sini: https://www.typescriptlang.org/tsconfig#jsx

Modul ES6

TypeScript mendukung ES6 (ECMAScript 2015) dan banyak versi berikutnya. Artinya, Anda dapat menggunakan sintaks ES6, seperti fungsi panah, literal templat, kelas, modul, destructuring, dan lainnya.

Untuk mengaktifkan fitur ES6 dalam proyek, Anda dapat menentukan properti `target` di `tsconfig.json`.

Contoh konfigurasi:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

Operator Eksponensiasi ES7

Operator eksponensial (`**`) menghitung nilai yang diperoleh dengan memangkatkan operand pertama dengan operand kedua. Fungsinya serupa dengan `Math.pow()`, tetapi memiliki kemampuan tambahan untuk menerima BigInt sebagai operand. TypeScript sepenuhnya mendukung operator ini dengan menetapkan `target` dalam berkas `tsconfig.json` ke `es2016` atau lebih tinggi.

```
console.log(2 ** (2 ** 2)); // 16
```

Pernyataan for-await-of

Ini adalah fitur JavaScript yang didukung sepenuhnya di TypeScript dan memungkinkan Anda melakukan iterasi atas objek iterable asinkron dengan versi target `es2018`.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

Meta-properti new target

Anda dapat menggunakan meta-properti `new.target` dalam TypeScript, yang memungkinkan Anda menentukan apakah suatu fungsi atau konstruktor dipanggil menggunakan operator `new`. Meta-properti ini memungkinkan Anda mendeteksi apakah sebuah objek dibuat sebagai hasil dari pemanggilan konstruktor.

```
class Parent {
    constructor() {
            console.log(new.target);  // Logs the constructor
function used to create an instance
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}
```

```
const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

Ekspresi Import Dinamis

Modul dapat dimuat secara kondisional atau secara malas sesuai permintaan menggunakan proposal impor dinamis ECMAScript yang didukung di TypeScript.

Sintaks untuk ekspresi impor dinamis dalam TypeScript adalah sebagai berikut:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
          const widget = await import('./widget'); // Dynamic
import
        widget.render(container);
    }
}

renderWidget();
```

“tsc –watch”

Perintah ini menjalankan kompiler TypeScript dengan parameter `--watch`, sehingga berkas TypeScript dapat dikompilasi ulang secara otomatis setiap kali diubah.

```
tsc --watch
```

Sejak TypeScript versi 4.9, pemantauan berkas terutama mengandalkan peristiwa sistem berkas dan secara otomatis beralih ke polling jika pemantau berbasis peristiwa tidak dapat dibuat.

Operator Asersi Non-null

Operator asersi non-null (postfix `!`), yang juga disebut asersi penetapan pasti, adalah fitur TypeScript yang memungkinkan Anda menegaskan bahwa variabel atau properti bukan `null` atau `undefined`, meskipun analisis tipe statis TypeScript menunjukkan bahwa nilainya mungkin demikian. Dengan fitur ini, pemeriksaan eksplisit dapat dihilangkan.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

Deklarasi dengan nilai default

Deklarasi dengan nilai default digunakan ketika variabel atau parameter diberi nilai default. Artinya, jika tidak ada nilai yang diberikan untuk variabel atau parameter tersebut, nilai default akan digunakan.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

Optional Chaining

Operator optional chaining `?.` bekerja seperti operator titik biasa (`.`) untuk mengakses properti atau metode. Namun, operator ini menangani nilai null atau undefined dengan baik,

yaitu dengan mengakhiri ekspresi dan mengembalikan `undefined`, alih-alih melempar galat.

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

Operator Nullish Coalescing

Operator nullish coalescing `??` mengembalikan nilai di sisi kanan jika sisi kiri bernilai `null` atau `undefined`; jika tidak, operator ini mengembalikan nilai di sisi kiri.

```
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

Tipe Template Literal

Tipe literal templat memungkinkan Anda memanipulasi nilai string pada tingkat tipe dan menghasilkan tipe string baru

berdasarkan tipe yang sudah ada. Tipe ini berguna untuk membuat tipe yang lebih ekspresif dan presisi dari operasi berbasis string.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

Overload fungsi

Overload fungsi memungkinkan Anda mendefinisikan beberapa signature fungsi untuk nama fungsi yang sama, masing-masing dengan tipe parameter dan tipe kembalian yang berbeda. Saat Anda memanggil fungsi yang di-overload, TypeScript menggunakan argumen yang diberikan untuk menentukan signature fungsi yang benar:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

Tipe Rekursif

Tipe rekursif adalah tipe yang dapat merujuk pada dirinya sendiri. Tipe ini berguna untuk mendefinisikan struktur data yang memiliki struktur hierarkis atau rekursif (dengan kemungkinan bersarang tanpa batas), seperti linked list, tree, dan graph.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

Tipe Kondisional Rekursif

Kita dapat mendefinisikan relasi tipe yang kompleks menggunakan logika dan rekursi dalam TypeScript. Mari kita uraikan dengan istilah sederhana:

Tipe kondisional memungkinkan Anda mendefinisikan tipe berdasarkan kondisi boolean:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

Rekursi berarti definisi tipe yang merujuk pada dirinya sendiri di dalam definisinya:

```
type Json = string | number | boolean | null | Json[] | { [key: string]: Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
        prop4: [],
```

```
    },
};
```

Tipe kondisional rekursif menggabungkan logika kondisional dan rekursi. Artinya, sebuah definisi tipe dapat bergantung pada dirinya sendiri melalui logika kondisional, sehingga menciptakan relasi tipe yang kompleks dan fleksibel.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 |
1 | 6
```

Dukungan Modul ECMAScript di Node

Node.js menambahkan dukungan untuk modul ECMAScript mulai versi 15.3.0, dan TypeScript telah memiliki dukungan modul ECMAScript untuk Node.js sejak versi 4.7. Dukungan ini dapat diaktifkan dengan menggunakan properti `module` bernilai `nodenext` dalam berkas `tsconfig.json`. Berikut contohnya:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js mendukung dua ekstensi berkas untuk modul: `.mjs` untuk modul ES dan `.cjs` untuk modul CommonJS. Ekstensi berkas yang setara dalam TypeScript adalah `.mts` untuk modul ES dan `.cts` untuk modul CommonJS. Ketika kompiler

TypeScript melakukan transpilasi berkas-berkas ini ke JavaScript, kompiler akan membuat berkas `.mjs` dan `.cjs`.

Jika ingin menggunakan modul ES dalam proyek, Anda dapat mengatur properti `type` menjadi `"module"` dalam berkas `package.json`. Hal ini menginstruksikan Node.js untuk memperlakukan proyek tersebut sebagai proyek modul ES.

Selain itu, TypeScript juga mendukung deklarasi tipe dalam berkas `.d.ts`. Berkas deklarasi ini menyediakan informasi tipe untuk pustaka atau modul yang ditulis dalam TypeScript, sehingga pengembang lain dapat memanfaatkannya dengan fitur pemeriksaan tipe dan pelengkapan otomatis TypeScript.

Fungsi Asersi

Dalam TypeScript, fungsi asersi adalah fungsi yang menunjukkan bahwa suatu kondisi tertentu telah diverifikasi berdasarkan nilai kembaliannya. Dalam bentuk paling sederhana, fungsi `assert` memeriksa predikat yang diberikan dan memunculkan galat ketika predikat tersebut bernilai `false`.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

Atau, fungsi tersebut dapat dideklarasikan sebagai ekspresi fungsi:

```
type AssertIsNumber = (value: unknown) => asserts value is
number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
```

```
        throw new Error('Not a number');
    }
};
```

Fungsi asersi memiliki kesamaan dengan type guard. Type guard pada awalnya diperkenalkan untuk melakukan pemeriksaan saat runtime dan memastikan tipe suatu nilai dalam cakupan tertentu. Secara khusus, type guard adalah fungsi yang mengevaluasi predikat tipe dan mengembalikan nilai boolean yang menunjukkan apakah predikat tersebut `true` atau `false`. Hal ini sedikit berbeda dari fungsi asersi, yang dimaksudkan untuk memunculkan galat alih-alih mengembalikan `false` ketika predikat tidak terpenuhi.

Contoh type guard:

```
const isNumber = (value: unknown): value is number => typeof
value === 'number';
```

Tipe Tuple Variadik

Tipe tuple variadik adalah fitur yang diperkenalkan dalam TypeScript versi 4.0. Jadi, mari kita mulai dengan meninjau kembali apa itu tuple:

Tipe tuple adalah array dengan panjang yang telah ditentukan, dan tipe setiap elemennya diketahui:

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

Istilah “variadik” berarti aritas tak tentu (menerima jumlah argumen yang bervariasi).

Tuple variadik adalah tipe tuple yang memiliki semua properti seperti sebelumnya, tetapi bentuk persisnya belum ditentukan:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

Dalam kode sebelumnya, kita dapat melihat bahwa bentuk tuple ditentukan oleh generik `T` yang diberikan.

Tuple variadik dapat menerima beberapa generik, sehingga sangat fleksibel:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

Dengan tuple variadik baru, kita dapat menggunakan:

- Elemen spread dalam sintaks tipe tuple kini dapat berupa generik, sehingga kita dapat merepresentasikan operasi tingkat tinggi pada tuple dan array bahkan ketika kita tidak mengetahui tipe sebenarnya yang sedang kita operasikan.
- Elemen rest dapat muncul di posisi mana pun dalam tuple.

Contoh:

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
```

```
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

Tipe Boxed

Tipe boxed merujuk pada objek pembungkus yang digunakan untuk merepresentasikan tipe primitif sebagai objek. Objek pembungkus ini menyediakan fungsionalitas dan metode tambahan yang tidak tersedia secara langsung pada nilai primitif.

Ketika Anda mengakses metode seperti `charAt` atau `normalize` pada primitif `string`, JavaScript membungkusnya dalam objek `String`, memanggil metode tersebut, lalu membuang objek itu.

Demonstrasi:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript merepresentasikan perbedaan ini dengan menyediakan tipe terpisah untuk tipe primitif dan objek pembungkus yang sesuai:

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol

- bigint => BigInt

Tipe boxed biasanya tidak diperlukan. Hindari penggunaan tipe boxed dan gunakan tipe primitif sebagai gantinya, misalnya `string` alih-alih `String`.

Kovarians dan Kontravarians dalam TypeScript

Kovarians dan kontravarians menjelaskan hubungan tipe dalam tipe generik. `Dog` adalah `Animal`, tetapi tidak semua `Animal` adalah `Dog`.

Kovarians (menghasilkan nilai): fungsi yang mengembalikan `Dog` dapat menggantikan fungsi yang mengembalikan `Animal`. Semua anjing adalah hewan, tetapi fungsi yang mengembalikan hewan apa pun bisa mengembalikan kucing.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;

produceAnimal = produceDog; // Valid
produceDog = produceAnimal; // Error
```

Kontravarians (menerima nilai): fungsi yang menerima semua `Animal` dapat menggantikan fungsi yang menerima `Dog`.

Fungsi itu juga dapat menangani anjing; fungsi yang hanya menerima anjing tidak aman untuk semua hewan.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // Valid
consumeAnimal = consumeDog; // Error with strictFunctionTypes
```

Panah menunjukkan arah penugasan yang aman:

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)
- Consumer<Animal> -> Consumer<Dog> (contravariance)

Array yang dapat diubah: TypeScript juga mengizinkan `Dog[]` ditetapkan ke `Animal[]`. Kedua variabel merujuk array yang sama; melalui tipe yang lebih luas, `Animal` yang bukan anjing dapat ditambahkan.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Allowed, but unsafe
// dogs now contains an Animal that is not a Dog.
```

Opsi compiler: dengan `strictFunctionTypes`, parameter fungsi mandiri diperiksa secara kontravarian. Tanpanya, TypeScript dapat mengizinkan kedua arah (bivarians), meskipun tidak aman. Parameter dalam deklarasi metode dan konstruktor merupakan pengecualian.

Anotasi Varians Opsional untuk Parameter Tipe

Sejak TypeScript 4.7, `out` menandai kovarians, `in` kontravarians, dan `in out` invarians. Biasanya TypeScript menyimpulkan varians. Anotasi harus sesuai dengan penggunaan parameter generik, bukan mengubah perilakunya secara sembarang.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

Index Signature Pola Template String

Index signature pola template string memungkinkan kita mendefinisikan index signature yang fleksibel menggunakan pola template string. Fitur ini memungkinkan kita membuat objek yang dapat diindeks dengan pola kunci string tertentu, sehingga memberikan kontrol dan kekhususan lebih besar ketika mengakses dan memanipulasi properti.

Mulai versi 4.4, TypeScript memungkinkan index signature untuk simbol dan pola template string.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
    'key-b': 456,
};
```

```
console.log(obj[uniqueSymbol]); // Unique symbol key
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

Operator satisfies

Operator `satisfies` memungkinkan Anda memeriksa apakah tipe tertentu memenuhi antarmuka atau kondisi tertentu. Dengan kata lain, operator ini memastikan bahwa suatu tipe memiliki semua properti dan metode yang diperlukan oleh antarmuka tertentu. Ini adalah cara untuk memastikan sebuah variabel sesuai dengan definisi suatu tipe. Berikut contohnya:

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Type Annotation using `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// In the following lines, TypeScript won't be able to infer
properly
user.attributes?.map(console.log); // Property 'map' does not
exist on type 'string | string[]'. Property 'map' does not
exist on type 'string'.
user.nickName; // string | string[] | undefined

// Type assertion using `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;
```

```
// Here too, TypeScript won't be able to infer properly
user2.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[]'. Property 'map' does not exist on type 'string'.
user2.nickName; // string | string[] | undefined

// Using the `satisfies` operator we can properly infer the types now
const user3 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log);   //   TypeScript   infers correctly: string[]
user3.nickName; // TypeScript infers correctly: undefined
```

Import dan Export Khusus Tipe

Impor dan ekspor khusus tipe memungkinkan Anda mengimpor atau mengekspor tipe tanpa mengimpor atau mengekspor nilai atau fungsi yang terkait dengan tipe tersebut. Hal ini dapat berguna untuk mengurangi ukuran bundel Anda.

Untuk menggunakan impor khusus tipe, Anda dapat menggunakan kata kunci `import type`.

TypeScript mengizinkan penggunaan ekstensi berkas deklarasi dan implementasi (`.ts`, `.mts`, `.cts`, dan `.tsx`) dalam impor khusus tipe, terlepas dari pengaturan `allowImportingTsExtensions`.

Sebagai contoh:

```
import type { House } from './house.ts';
```

Bentuk-bentuk berikut didukung:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

Deklarasi using dan Pengelolaan Sumber Daya Eksplisit

Deklarasi `using` adalah binding imutabel dengan cakupan blok, serupa dengan `const`, yang digunakan untuk mengelola sumber daya yang dapat di-dispose. Ketika diinisialisasi dengan sebuah nilai, metode `Symbol.dispose` dari nilai tersebut dicatat dan kemudian dijalankan saat keluar dari cakupan blok yang mengapitnya.

Hal ini didasarkan pada fitur Pengelolaan Sumber Daya ECMAScript, yang berguna untuk menjalankan tugas pembersihan penting setelah objek dibuat, seperti menutup koneksi, menghapus berkas, dan membebaskan memori.

Catatan:

- Karena baru diperkenalkan dalam TypeScript versi 5.2, sebagian besar runtime belum memiliki dukungan bawaan. Anda akan memerlukan polyfill untuk: `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack`, `SuppressedError`.
- Selain itu, Anda perlu mengonfigurasi `tsconfig.json` sebagai berikut:

```
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

Contoh:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Simple polyfill

const doWork = (): Disposable => {
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // Resource is declared
    console.log(2);
} // Resource is disposed (e.g., `work[Symbol.dispose]()` is
evaluated)

console.log(3);
```

Kode tersebut akan menampilkan:

```
1
2
disposed
3
```

Sumber daya yang dapat di-dispose harus mematuhi antarmuka `Disposable`:

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

Deklarasi `using` mencatat operasi disposal sumber daya dalam sebuah stack, sehingga sumber daya di-dispose dalam urutan deklarasi terbalik:

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // disposes `C`, then `B`, then `A`.
```

Sumber daya dijamin akan di-dispose, bahkan jika kode berikutnya dijalankan atau terjadi exception. Hal ini dapat menyebabkan proses disposal memunculkan exception yang mungkin menyembunyikan exception lainnya. Untuk mempertahankan informasi tentang galat yang ditekan, exception bawaan baru, `SuppressedError`, diperkenalkan.

Deklarasi await using

Deklarasi `await using` menangani sumber daya yang dapat di-dispose secara asinkron. Nilainya harus memiliki metode `Symbol.asyncDispose`, yang akan ditunggu dengan `await` pada akhir blok.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // Resource is declared
```

```
}     //     Resource     is     disposed     (e.g.,     `await
work[Symbol.asyncDispose]()` is evaluated)
```

Sumber daya yang dapat di-dispose secara asinkron harus mematuhi antarmuka `Disposable` atau `AsyncDisposable`:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose   ??=   Symbol('Symbol.asyncDispose');   //
Simple polyfill

class DatabaseConnection implements AsyncDisposable {
     // A method that is called when the object is disposed
asynchronously
    [Symbol.asyncDispose]() {
        // Close the connection and return a promise
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
             await  new  Promise(resolve  =>  setTimeout(resolve,
1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
     // Create a new connection and dispose it asynchronously
when it goes out of scope
      await  using  connection  =  new  DatabaseConnection();  //
Resource is declared
    console.log('Doing some work...');
}     //     Resource     is     disposed     (e.g.,     `await
connection[Symbol.asyncDispose]()` is evaluated)
```

```
doWork();
```

Kode tersebut menampilkan:

```
Doing some work...
Closing the connection...
Connection closed.
```

Deklarasi `using` dan `await using` diizinkan dalam pernyataan: `for`, `for-in`, `for-of`, `for-await-of`, `switch`.

Atribut Import

Atribut impor TypeScript 5.3 (label untuk impor) memberi tahu runtime cara menangani modul (JSON, dan sebagainya). Hal ini meningkatkan keamanan dengan memastikan impor yang jelas dan selaras dengan Content Security Policy (CSP) untuk pemuatan sumber daya yang lebih aman. TypeScript memastikan atribut tersebut valid, tetapi menyerahkan interpretasinya kepada runtime untuk penanganan modul tertentu.

Contoh:

```
import config from './config.json' with { type: 'json' };
```

dengan impor dinamis:

```
const config = import('./config.json', { with: { type: 'json' }
});
```

Pemeriksaan Sintaks Ekspresi Reguler

Sejak TypeScript 5.5.4, TypeScript memeriksa literal regex untuk galat umum pada saat kompilasi (misalnya sintaks yang tidak valid, backreference yang salah, dan fitur yang tidak didukung untuk versi JS target Anda). Hal ini membantu menemukan bug lebih awal, tetapi tidak memeriksa string `new RegExp("...")`.

```
let r = /(a)\2/; // Error: This backreference refers to a group that does not exist.
```

import defer

`import defer` memungkinkan Anda memuat modul tetapi menunda eksekusinya hingga Anda benar-benar menggunakan sesuatu dari modul tersebut. Hal ini membantu menghindari pekerjaan dan efek samping yang tidak diperlukan.

- Hanya berfungsi dengan: `import defer * as name from "module"`
- Kode hanya berjalan ketika Anda mengakses ekspor